

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)

на тему:

**«Інформаційна система обліку та моніторингу ІТ-
активів підприємства»**

Виконав студент групи ПЗ-22-3
БУРКІВСЬКИЙ Сергій Вадимович

Керівник роботи:
СУГОНЯК Інна Іванівна

Рецензент:
ВЛАСЕНКО Олег Васильович

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»

Зав. кафедри інженерії
програмного забезпечення

Тетяна Вакалюк

«28» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: **БУРКІВСЬКИЙ Сергій Вадимович**

Керівник роботи: **СУГОНЯК Інна Іванівна**

Тема роботи: **«Інформаційна система обліку та моніторингу ІТ-активів підприємства»**,

затверджена Наказом закладу вищої освіти від **«27» лютого 2026 р., № 92/с**

Вихідні дані для роботи: Об'єкт дослідження – процес збору, систематизації, моніторингу та управління життєвим циклом ІТ-активів і програмного забезпечення на підприємстві; Предмет дослідження – алгоритми контролю станів обладнання, архітектурні рішення веб-орієнтованої системи та технологічний стек розробки платформи для автоматизації обліку та аудиту ІТ-ресурсів. Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	Консультант	Завдання видав	Завдання прийняв
1	Сугоняк І.І.	01.03.2026р.	01.03.2026р
2	Сугоняк І.І.	01.03.2026р	01.03.2026р
3	Сугоняк І.І.	01.03.2026р	01.03.2026р

РЕФЕРАТ

Пояснювальна записка до випускної кваліфікаційної роботи бакалавра на тему “Інформаційна система обліку та моніторингу ІТ-активів підприємства” містить 75 сторінок друкованого тексту. Робота ілюстрована 38 рисунками та містить 18 таблиць. Перелік використаних джерел налічує 31 найменувань і займає 3 сторінок. Пояснювальна записка також містить 26 сторінок додатків. Загальний обсяг роботи становить 101 сторінок.

Об’єкт дослідження процес збору, систематизації, моніторингу та управління життєвим циклом ІТ-активів і програмного забезпечення на підприємстві.

Предмет дослідження алгоритми контролю станів обладнання, архітектурні рішення веб-орієнтованої системи та технологічний стек розробки платформи для автоматизації обліку та аудиту ІТ-ресурсів.

Мета роботи підвищення ефективності управління матеріально-технічною базою підприємства шляхом розробки та впровадження інформаційної системи, що забезпечує автоматизований облік, контроль термінів обслуговування та мінімізацію ризиків втрати ІТ-активів.

У роботі використано методи порівняльного аналізу для огляду існуючих рішень, системного аналізу для формулювання вимог до системи, методи об’єктно-орієнтованого проектування та моделювання баз даних.

Практичне значення результатів полягає у розробці готового програмного продукту. Створена інформаційна система обліку ІТ-активів побудована за клієнт-серверною архітектурою. Система містить інструменти для контролю станів обладнання, управління ліцензіями та формування автоматизованої звітності. Використано сучасний стек вебтехнологій (Next.js [20], TypeScript [10, 29], Prisma, PostgreSQL [22]) для забезпечення надійності збереження даних та швидкодії системи.

					ІПЗ.КР.Б – 121 – 26 – ПЗ			
Змн.	Арк.	№ докум.	Підпис	Дата	Інформаційна система обліку та моніторингу стану ІТ-активів підприємства	Літ.	Арк.	Аркуші
Розроб.		Бурківський С.В.						
Керівник		Сугоняк І.І.					3	101
Рецензент		Власенко О.В.				Житомирська політехніка, група ІПЗ-22-3		
Зав. каф		Вакалюк Т.А.						

Створена платформа надає адміністраторам інструментарій для оперативного аудиту, фільтрації та глибокого аналізу стану технічного забезпечення підприємства.

КЛЮЧОВІ СЛОВА: ІНФОРМАЦІЙНА СИСТЕМА, ОБЛІК ІТ-АКТИВІВ, МОНІТОРИНГ СТАНУ, ВЕБЗАСТОСУНОК, КЛІЄНТ-СЕРВЕРНА АРХІТЕКТУРА, NEXT.JS, PRISMA, БАЗА ДАНИХ.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						4
Змн.	Арк.	№ докум.	Підпис	Дата		

ABSTRACT

The explanatory note to the bachelor's graduation qualification thesis on the topic "Information System for Accounting and Monitoring IT Assets of an Enterprise" contains 75 pages of printed text. The work is illustrated with 38 figures and includes 18 tables. The list of references comprises 31 sources and occupies 3 pages. The explanatory note also contains 26 pages of appendices. The total volume of the work is 101 pages.

The object of the study is the process of collecting, systematizing, monitoring, and managing the lifecycle of IT assets and software within an enterprise.

The subject of the study is algorithms for equipment state monitoring, architectural solutions of a web-based system, and the technology stack for developing a platform for automating accounting and auditing of IT resources.

The aim of the work is to improve the efficiency of managing the material and technical base of an enterprise by developing and implementing an information system that provides automated accounting, maintenance scheduling control, and minimization of risks related to IT asset loss.

The research methods used include comparative analysis for reviewing existing solutions, system analysis for defining system requirements, object-oriented design methods, and database modeling techniques.

The practical significance of the results lies in the development of a ready-to-use software product. The developed IT asset accounting information system is built on a client-server architecture. The system includes tools for monitoring equipment status, managing licenses, and generating automated reports. A modern web technology stack (Next.js [20], TypeScript [10, 29], Prisma, PostgreSQL [22]) is used to ensure reliable data storage and system performance.

The developed platform provides administrators with tools for operational auditing, filtering, and in-depth analysis of the enterprise's technical infrastructure.

KEYWORDS: INFORMATION SYSTEM, IT ASSET ACCOUNTING, STATUS MONITORING, WEB APPLICATION, CLIENT-SERVER ARCHITECTURE, NEXT.JS, PRISMA, DATABASE.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

ЗМІСТ

РЕФЕРАТ.....	3
ABSTRACT	5
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	8
ВСТУП.....	9
РОЗДІЛ 1. ПРЕДМЕТНА ОБЛАСТЬ ТА ПОСТАНОВКА ЗАДАЧІ.....	12
1.1 Постановка задачі та огляд існуючих систем-аналогів.....	12
1.2. Функціональні та нефункціональні вимоги до системи.	16
1.3. Вибір стеку технологій та апаратного забезпечення	19
1.4. Інструментальні засоби розробки.....	22
Висновки до першого розділу	25
РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ.....	27
2.1 Моделювання варіантів використання та поведінки системи	27
2.2. Проєктування логічної структури бази даних.....	33
2.3. Проєктування компонентної структури та взаємодії модулів	38
2.4. Розробка математичної моделі та алгоритмів обробки даних в системі	43
Висновки до другого розділу	48
Розділ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ, ІНТЕРФЕЙС ТА ТЕСТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ УПРАВЛІННЯ ІТ-АКТИВАМИ.....	50
3.1 Порядок встановлення та налаштування параметрів системи	50
3.2 Структура інтерфейсу інформаційної системи моніторингу та управління ІТ-активами	54
3.3 Тестування роботи програмного продукту	65
Висновки до третього розділу	69
ВИСНОВКИ	71

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ..... 73

ДОДАТКИ 76

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД – база даних.

ЗІ – засіб інвентаризації.

ІЗ – інформаційні заходи.

ІС – інформаційна система.

ІТ – інформаційні технології.

ПЗ – програмне забезпечення.

ПК – персональний комп'ютер.

СУБД – система управління базами даних.

ТЗ – технічне завдання.

API (Application Programming Interface) – інтерфейс програмування додатків.

CSS (Cascading Style Sheets) – каскадні таблиці стилів.

ER-діаграма (Entity-Relationship diagram) – діаграма «сутність-зв'язок».

HTML (HyperText Markup Language) – мова розмітки гіпертексту.

HTTP (HyperText Transfer Protocol) – протокол передачі гіпертексту.

IDE (Integrated Development Environment) – інтегроване середовище розробки.

JSON (JavaScript Object Notation) – текстовий формат обміну даними, що використовується в API.

ORM (Object-Relational Mapping) – об'єктно-реляційне відображення.

RBAC (Role-Based Access Control) – керування доступом на основі ролей.

REST (Representational State Transfer) – архітектурний стиль взаємодії компонентів розподіленого додатка.

SQL (Structured Query Language) – мова структурованих запитів.

SSR (Server-Side Rendering) – рендеринг (візуалізація) вебсторінок на стороні сервера.

UML (Unified Modeling Language) – уніфікована мова моделювання.

URL (Uniform Resource Locator) – уніфікований показник ресурсу.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						8
Змн.	Арк.	№ докум.	Підпис	Дата		

ВСТУП

Актуальність теми. Сучасний етап розвитку інформаційних технологій характеризується цифровізацією всіх сфер управління ресурсами, і сфера ІТ-менеджменту не є винятком. Щодня в межах діяльності підприємств генеруються та змінюються величезні масиви даних про стан інфраструктури: технічні характеристики обладнання, терміни дії ліцензій, історія сервісного обслуговування та переміщення активів. Традиційні підходи до збору та аналізу такої інформації, які часто базуються на використанні розрізнених джерел або універсальних електронних таблиць, є неефективними. Цей процес вимагає значних витрат часу на ручну обробку, підвищує ризик помилок і не дозволяє оперативно отримувати цілісну картину для прийняття стратегічних рішень щодо модернізації чи закупівель.

В умовах високої потреби в оптимізації витрат ІТ-директорам, системним адміністраторам та менеджерам потрібні швидкі, гнучкі та точні інструменти для роботи з інвентаризаційними даними. Саме тому в останні роки спостерігається стійкий тренд на впровадження спеціалізованих платформ для управління активами (IT Asset Management). Розробка профільних інформаційних систем дозволяє автоматизувати процеси збору, агрегації та моніторингу стану техніки. Надання користувачам зручних інтерактивних дашбордів та автоматизованих звітів значно зменшує час на пошук потрібної інформації та знижує ризики втрати активів.

Але розробка сучасних систем моніторингу ІТ-активів є складною архітектурною задачею. Вона вимагає проєктування надійних баз даних для збереження великих обсягів інформації та реалізації оптимізованих алгоритмів обробки запитів. Побудова системи вимагає впровадження ефективних підходів до структурування коду, наприклад, клієнт-серверної архітектури на основі компонентного підходу та API, що дозволяє логічно розділити предметні області (обладнання, персонал, програмне забезпечення) без зайвого ускладнення інфраструктури. Поєднання сучасного стеку технологій для клієнтської та серверної частин забезпечує необхідну продуктивність, інтерактивність та

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						9
Змн.	Арк.	№ докум.	Підпис	Дата		

швидкість відгуку системи. З огляду на це, проектування та розробка спеціалізованої інформаційної системи обліку та моніторингу стану ІТ-активів, що базується на сучасних архітектурних патернах, є актуальним науково-практичним завданням.

Об'єкт дослідження процес проектування, розробки та функціонування інформаційних систем для обробки, збереження та управління даними про матеріально-технічні ресурси.

Предмет дослідження методи, архітектурні підходи та програмні засоби створення інформаційної системи обліку та моніторингу стану ІТ-активів.

Мета роботи полягає у проектуванні та розробці програмної системи для збору, агрегації, зберігання та візуалізації інформації про стан ІТ-інфраструктури, яка забезпечує автоматизацію облікових процесів та надає зручний інструментарій для моніторингу життєвого циклу активів.

Для досягнення поставленої мети було сформульовано та вирішено такі завдання дослідження, проаналізувати предметну область, дослідити особливості управління ІТ-інфраструктурою та провести огляд існуючих інформаційних систем-аналогів.

Обґрунтувати вибір сучасного стеку вебтехнологій та інструментів розробки, визначити функціональні й нефункціональні вимоги до програмного продукту.

Спроекувати загальну архітектуру системи з використанням клієнт-серверного підходу, побудувати об'єктно-орієнтовану модель та описати структуру взаємодії компонентів.

Спроекувати реляційну базу даних для збереження показників обладнання та ліцензій, розробити алгоритми обробки інформації та розмежування прав доступу.

Реалізувати інформаційну систему моніторингу активів, включаючи розробку серверної частини (API), клієнтського інтерфейсу з дашбордами, модулів обліку та панелі адміністратора.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						10
Змн.	Арк.	№ докум.	Підпис	Дата		

Провести тестування системи, налаштувати механізми безпеки даних та здійснити перевірку працездатності готового програмного продукту.

Методи дослідження. У роботі використано методи порівняльного та системного аналізу для огляду існуючих рішень і формування вимог до програмного забезпечення, методи об'єктно-орієнтованого проєктування (UML) для розробки структури системи, а також методи моделювання реляційних баз даних.

Практичне значення одержаних результатів. Створена інформаційна система обліку та моніторингу стану ІТ-активів надає користувачам інтуїтивно зрозумілий інструментарій для інвентаризації, контролю гарантійних термінів та візуалізації стану інфраструктури. Це знижує складність аналізу великих масивів інформації та оптимізує часові й ресурсозатрати системних адміністраторів та менеджерів. Архітектурні рішення, закладені в систему, дозволяють масштабувати платформу для потреб підприємств різного розміру та інтегрувати її з іншими корпоративними сервісами.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						11
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Постановка задачі та огляд існуючих систем-аналогів

Предметною областю даного дипломного проєкту є сфера управління IT-інфраструктурою (IT Asset Management ITAM [12]), а саме сегмент спеціалізованих інформаційних систем, які надають можливість збирати, обробляти, зберігати та моніторити дані про апаратні та програмні ресурси підприємства.

Розвиток цифровізації бізнесу диктує нові правила: використання об'єктивних метрик для оцінки стану парку техніки, контролю ліцензійної чистоти ПЗ та прогнозування витрат на модернізацію стало базовою вимогою для досягнення високої конкурентоспроможності. Проте ефективне управління цими масивами даних традиційними методами стикається з низкою суттєвих перешкод.

Традиційний процес обліку IT-активів є складним, багатоетапним та ресурсомістким циклом. Зазвичай для підготовки детального аналізу стану інфраструктури потрібне залучення фахівців, які вручну збирають інформацію з розрізнених джерел. Спочатку дані заносяться у звичайні табличні процесори (наприклад, Microsoft Excel). Далі системні адміністратори намагаються звести цю інформацію в єдину структуру, проводять розрахунки термінів амортизації та вручну будують звіти для керівництва.

Такий класичний підхід є неоптимальним, дорогим і тривалим. Для сучасних технологічних компаній витрати часу на ручну обробку великих масивів даних про сотні одиниць техніки є критичними. Крім того, будь-яка потреба змінити параметри аналізу наприклад, відфільтрувати техніку за періодом експлуатації, порівняти показники зносу різних типів обладнання або перевірити терміни дії ліцензій вимагає повторного ручного перерахунку. Це створює затримки в прийнятті рішень, підвищує ризик виникнення помилок через людський фактор і значно ускладнює роботу з інформацією.

Усвідомлення цих бар'єрів призвело до еволюції інструментарію IT-менеджменту та появи концепції інтегрованих платформ. Основною ідеєю цього

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						12
Змн.	Арк.	№ докум.	Підпис	Дата		

підходу є абстрагування складності роботи з «сирими» даних: рутинні технічні процеси розрахунків, фільтрації та агрегації перекладаються з людини на програмні алгоритми. Замість роботи з неструктурованими таблицями користувач взаємодіє зі зрозумілим графічним інтерфейсом, візуально оцінюючи ключові метрики через інтерактивні дашборди.

Важливою вимогою є забезпечення швидкодії, інтерактивності та адаптивності застосунку. Робота з великими обсягами технічних даних вимагає оптимізованої архітектури (зокрема, використання сучасної клієнт-серверної компонентної архітектури) для швидкої віддачі інформації. Програмний комплекс повинен самостійно вирішувати проблему масштабування інтерфейсу, надаючи віджетам попередньо налаштовану поведінку, яка гнучко перебудовує контент залежно від пристрою користувача.

Опираючись на проведений аналіз, головним завданням даної роботи є проєктування та розробка програмного продукту «Інформаційна система обліку та моніторингу стану ІТ-активів підприємства». Ця система повинна стати комплексним рішенням, де користувач без глибоких навичок роботи з базами даних зможе через зручний інтерфейс керувати повним життєвим циклом обладнання. Платформа має приховати технічну складність побудови SQL-запитів, надавши інтуїтивно зрозумілий інструментарій для моніторингу активів. Досягнення цієї мети вимагає дослідження архітектурних підходів, методів оптимізації реляційних баз даних та організації ефективної клієнт-серверної взаємодії.

Дослідження концепції візуальної аналітики та архітектурних підходів. Стрімкий розвиток ІТ-технологій спричинив поширення концепції Self-Service BI у сфері корпоративного управління. У контексті вебзастосунків для обліку активів це означає зміну внутрішньої архітектури: процес написання скриптів для вибірки даних замінюється взаємодією користувача з графічним інтерфейсом. Ключовим механізмом тут виступає принцип реактивного відображення даних оновлення стану активу в базі даних має миттєво

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						13
Змн.	Арк.	№ докум.	Підпис	Дата		

відображатися на всіх пов'язаних аналітичних віджетах без перезавантаження сторінки.

Для забезпечення гнучкості інтерфейсу доцільно застосувати концепцію Schema-Driven UI. Конфігурація кожного звіту або картки активу зберігається у форматі JSON [30], що дозволяє динамічно підключати потрібні компоненти залежно від типу обладнання. Такий підхід ідеально узгоджується з сучасними методологіями розробки інтерфейсів, де кожен елемент (графік зносу, таблиця ліцензій) є ізольованим компонентом.

Аналіз існуючих систем-аналогів для обґрунтування доцільності розробки власної системи. Було проведено огляд існуючих рішень на ринку ITAM-систем [12]. Як базові аналоги обрано платформи Snipe-IT та GLPI.

Платформа Snipe-IT потужна система з відкритим кодом для обліку активів. Головною перевагою є наявність детальних карток обладнання та системи штрих-кодування. Проте архітектура системи орієнтована на статичні звіти, що обмежує можливості кастомізації дашбордів під специфічні потреби конкретного підприємства.

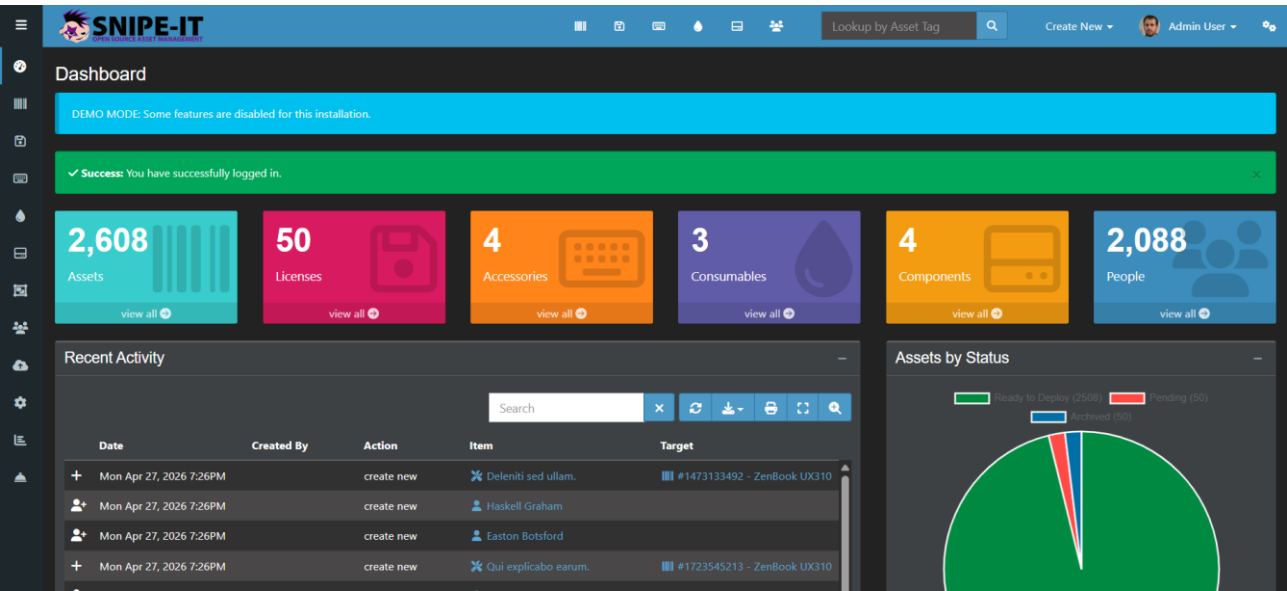


Рис 1.1 Інтерфейс вебзастосунку Snipe-IT

Платформа GLPI глобальна система, що поєднує облік активів та Service Desk. Вона здатна обробляти величезні масиви даних, проте її інтерфейс є надмірно складним і перевантаженим функціями, які не використовуються малими та середніми підприємствами.



Рис 1.2 Інтерфейс вебзастосунку GLPI

Для наочної систематизації результатів аналізу існуючих аналогів та обґрунтування функціональних і нефункціональних вимог до власної розробки, основні критерії порівняння зведено у таблицю (табл. 1.1).

Таблиця 1.1

Порівняння інформаційних систем обліку ІТ-активів

Критерії порівняння	Snipe-IT	GLPI	Платформа, що розробляється
Кастомізація інтерфейсу	Ні	Ні	Так
Формат збереження конфігурацій	Закритий	Закритий	Відкритий
Конструктор аналітичних звітів	Частково	Ні	Так
Наявність інтерактивних дашбордів	Частково	Ні	Так

Моніторинг життєвого циклу	Так	Так	Так
Архітектурна модульність	Середня	Висока	Висока

Проведений аналіз показує, що існуючі рішення успішно вирішують завдання агрегації даних, проте їхня архітектура передбачає статичне розміщення елементів. Вони не надають користувачеві інструментарію для гнучкого конструювання звітів «під себе». Це підтверджує актуальність розробки власної системи, яка поєднає потужну базу даних на серверній частині та гнучкий, керований конфігураціями інтерфейс.

1.2. Функціональні та нефункціональні вимоги до системи.

Аналіз цільової аудиторії та основних сценаріїв використання системи. Одним із ключових етапів проектування програмного продукту є визначення його цільової аудиторії. Розуміння потреб кінцевих користувачів дозволяє оптимізувати користувацький інтерфейс, покращити досвід взаємодії (UX) та сфокусувати зусилля на розробці найбільш затребуваного функціоналу. Аналіз бізнес-процесів управління IT-інфраструктурою дозволяє виділити три основні сегменти цільової аудиторії розроблюваної платформи, кожен з яких має власні болі та специфічні вимоги до системи.

Першим сегментом є системні адміністратори та IT-інженери. Це користувачі, яким необхідне швидке отримання актуальних технічних даних про кожну одиницю обладнання для оперативного вирішення інцидентів. Основна проблематика полягає в тому, що традиційний облік у ручних таблицях не дозволяє миттєво дізнатися конфігурацію ПК, термін дії гарантії або історію ремонтів без фізичного огляду техніки. Платформа вирішує цю проблему шляхом надання детальних цифрових паспортів активів. Інженер отримує можливість миттєво знаходити потрібний пристрій за серійним номером або QR-кодом, переглядати його поточний стан та закріпленого користувача через готові інформаційні картки.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						16
Змн.	Арк.	№ докум.	Підпис	Дата		

Другий сегмент аудиторії ІТ-менеджери та керівники відділів. Це професіонали, яким необхідно приймати стратегічні рішення щодо модернізації парку техніки та оптимізації бюджету. Їхньою головною проблемою є відсутність консолідованої аналітики: важко оцінити загальну вартість активів, виявити обладнання, що потребує списання, або відстежити ефективність використання ліцензій на ПЗ. Рішенням від розроблюваної платформи є надання гнучкого візуального конструктора дашбордів, який дозволяє менеджеру за принципом Drag-and-Drop самостійно формувати звіти про завантаженість складів, витрати на сервісне обслуговування та прогнозовані терміни оновлення застарілої техніки.

Третій сегмент складають бухгалтери та фахівці з логістики. Вони працюють з активами з точки зору фінансового обліку та інвентаризації. Часто вони стикаються з проблемою розбіжностей між фактичною наявністю техніки та записами в документах. На відміну від закритих або надто складних ERP-систем, розроблювана платформа підтримує функції гнучкого експорту та звітності. Користувач може сформувати інвентаризаційну відомість за певним відділом або категорією, а потім вивантажити її у вигляді структурованого CSV або PDF-файлу. При цьому реалізовано принцип «Single Source of Truth» (єдиного джерела істини), що гарантує будь-яка модифікація даних у звіті не впливає на цілісність глобальної бази даних системи.

На основі профілів цільової аудиторії сформовано базові сценарії взаємодії, які підтверджують здатність платформи покривати різні бізнес-потреби. Сценарій оперативного обліку та закріплення активу розпочинається з надходження нового обладнання на склад. Модератор даних (комірник) проходить авторизацію, створює нову картку активу, вносить технічні характеристики та завантажує фотофіксацію стану через Cloudinary [6]. Далі, використовуючи систему пошуку, він обирає співробітника, за яким закріплюється техніка. Система автоматично генерує запис про переміщення, оновлює статус активу на «В експлуатації» та реактивно перемальовує загальний дашборд наявності техніки на складі.

Сценарій формування аналітичного звіту про життєвий цикл має інший характер. Менеджер відкриває розділ конструктора звітів. За допомогою інтерактивних фільтрів він обирає категорію (наприклад, «Сервери») та параметр фільтрації (термін експлуатації > 5 років). Система миттєво відображає перелік критичного обладнання. Користувач ініціює процедуру експорту, і серверна частина генерує готовий графічний PDF-звіт із діаграмами зносу, який менеджер використовує для обґрунтування бюджету на закупівлю нової інфраструктури.

Функціональні вимоги описують поведінку системи та визначають перелік функцій для задоволення потреб підприємства. Архітектура базується на моделі управління доступом на основі ролей (RBAC). Детальний перелік можливостей наведено у таблиці 1.2.

Таблиця 1.2

Функціональні вимоги до системи за ролями користувачів

Роль користувача	Функціональні можливості у системі
Гість (Неавторизований)	Перегляд сторінки авторизації, захист від доступу до внутрішніх маршрутів системи (NextAuth).
Співробітник (User)	Перегляд власного профілю та списку техніки, закріпленої за ним. Створення тікетів у службі підтримки (Helpdesk) щодо несправності обладнання.
Технічний спеціаліст (Tech)	Повний цикл CRUD-операцій над ІТ-активами: додавання, редагування специфікацій, зміна статусів. Проведення операцій видачі/повернення техніки. Обробка та закриття тікетів служби підтримки. Моніторинг ліцензій ПЗ.
Адміністратор (Admin)	Повний доступ до всіх модулів. Керування ролями користувачів та системними налаштуваннями бази даних. Доступ до дашбордів, конструктора звітів та системних логів (аудит дій та подій безпеки).
Роль користувача	Функціональні можливості у системі

Нефункціональні вимоги описують атрибути якості системи, визначаючи інженерні метрики її роботи.

Вимоги до продуктивності та швидкодії визначають швидкість реакції системи. Час відгуку серверного API на запити вибірки активів не повинен перевищувати 250 мс. Оскільки інтерфейс побудований на React [24], рендеринг складних таблиць та дашбордів має відбуватися з частотою 60 FPS. Система повинна оптимізувати передачу JSON-даних [30] через стиснення та кешування результатів частих запитів на стороні клієнта.

Вимоги до надійності та відмовостійкості спираються на використання хмарної інфраструктури та сучасної архітектури на базі Next.js [20]. Цільовий показник доступності системи (uptime) 99,9%. База даних PostgreSQL [22] повинна підтримувати автоматичне щоденне резервне копіювання. У разі критичної помилки на сервері (наприклад, у середовищі Vercel [1]), механізм авто-відновлення має перезавантажити сервіс протягом 30-60 секунд без втрати даних.

Вимоги до безпеки є пріоритетними, оскільки система містить дані про внутрішню інфраструктуру. Весь трафік має бути захищений протоколом HTTPS. Паролі користувачів хешуються надійними алгоритмами (наприклад, bcrypt). Для захисту від автоматизованих атак інтегрується Cloudflare Turnstile. Використання ORM (Prisma) на сервері має повністю нівелювати ризики SQL-ін'єкцій при роботі з динамічними фільтрами.

Вимоги до програмного та апаратного забезпечення. Система повинна коректно працювати у сучасних браузерях (Chrome, Firefox, Safari, Edge). Апаратні вимоги до клієнта: мінімум 4 ГБ ОЗП (для комфортної роботи з дашбордами 8 ГБ) та стабільне інтернет-з'єднання від 5 Мбіт/с для швидкого завантаження JSON-конфігурацій [30] та зображень активів.

1.3. Вибір стеку технологій та апаратного забезпечення

Ефективність роботи інформаційної системи обліку та моніторингу IT-активів критично залежить від правильно обраного підходу до зберігання даних.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						19
Змн.	Арк.	№ докум.	Підпис	Дата		

Архітектурний виклик полягає у необхідності одночасно забезпечувати сувору транзакційну цілісність для базового реєстру техніки (інвентарні номери, серійні коди, фінансові операції) та максимальну гнучкість для збереження технічних характеристик обладнання. Оскільки різні типи активів мають абсолютно несхожі набори параметрів (наприклад, для сервера важливо зберігати кількість ядер та обсяг RAM, а для монітора діагональ та тип матриці), традиційні підходи до проєктування баз даних [16] з жорсткою схемою потребують перегляду. Під час проєктування було проаналізовано три основні патерни зберігання таких даних.

Метод 1, патерн EAV (Entity-Attribute-Value) у реляційних базах історично для зберігання динамічних атрибутів у SQL-базах використовувався патерн EAV. Згідно з цим підходом, створюються три таблиці: Сутність (Актив), Атрибут (Параметр) та Значення. Незважаючи на теоретичну гнучкість, цей підхід є антипатерном для систем обліку. Його головним недоліком є катастрофічне падіння продуктивності при читанні даних та складність написання запитів. Щоб завантажити специфікації лише одного робочого місця (ПК, монітор, периферія), базі даних доведеться виконати десятки операцій JOIN. Це створює надмірне навантаження на процесор сервера і робить швидкий пошук за технічними характеристиками майже неможливим.

Метод 2, документо-орієнтовані бази даних (NoSQL) Альтернативним рішенням є використання NoSQL баз даних, таких як MongoDB. У цій парадигмі опис активу зберігається у вигляді єдиного JSON-документа [30]. Це ідеально підходить для опису техніки, де набір полів може змінюватися. Однак використання виключно NoSQL створює критичні проблеми для платформи. Оскільки система оперує даними про матеріальну відповідальність, фінансові витрати та ліцензії, необхідна суворя підтримка ACID-транзакцій. Забезпечення цілісності складних зв'язків (наприклад, каскадне оновлення статусу всіх ліцензій при списанні сервера) у нереляційних базах вимагає написання складної програмної логіки на рівні бекенду, що підвищує ризик виникнення аномалій.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						20
Змн.	Арк.	№ докум.	Підпис	Дата		

Метод 3, гібридний підхід (PostgreSQL [22] з підтримкою JSONB [30]) Для вирішення цього архітектурного компромісу у розроблюваній системі застосовано гібридний підхід. В якості основної системи управління базами даних обрано реляційну СУБД PostgreSQL [22]. Вона гарантує цілісність критичних сутностей (користувачі, склади, типи активів) через класичні таблиці зі строгими зовнішніми ключами. Водночас PostgreSQL [22] надає потужну нативну підтримку типу даних JSONB [30]. Це дозволяє зберігати варіативні технічні характеристики активів у бінарному JSON-форматі [30] всередині відповідних колонок.

Переваги такого архітектурного рішення для платформи, швидкодія читання, сервер отримує повну інформацію про актив єдиним простим SQL-запитом без складних об'єднань.

Уникнення міграцій, додавання нових категорій обладнання з унікальними полями не вимагає критичної зміни схеми бази даних.

Індексація та пошук тип JSONB [30] у PostgreSQL [22] підтримує GIN-індекси, що дозволяє виконувати миттєвий пошук за значеннями всередині JSON-об'єктів [30].

Для забезпечення відмовостійкості, безпеки та безперервного розгортання платформи було обрано сучасні хмарні рішення, що дозволяють побудувати надійну архітектуру для корпоративного використання.

Як платформу для комплексного хостингу (фронтенду та серверних API-маршрутів) обрано Vercel [1]. Цей сервіс ідеально оптимізований для розгортання Fullstack-застосунків, розроблених на Next.js [20]. Він надає вбудовану систему CI/CD [11], автоматично налаштовує безпечне HTTPS-з'єднання, роздає статичні файли через глобальну мережу доставки контенту (CDN) та забезпечує миттєве завантаження візуальних компонентів системи у користувачів з будь-якого філіалу підприємства.

В якості хмарного провайдера для СУБД PostgreSQL [22] обрано платформу Aiven [2] (модель DBaaS). Це дозволяє уникнути витрат часу на ручне адміністрування серверів БД. Платформа бере на себе автоматичне резервне

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						21
Змн.	Арк.	№ докум.	Підпис	Дата		

копіювання реєстрів техніки, шифрування даних у спокої та забезпечення високої доступності бази даних.

Для управління медіафайлами інтегровано сервіс Cloudinary [6]. Зберігання графічних матеріалів безпосередньо в БД є неефективним, тому Cloudinary [6] автоматично оптимізує зображення, кадрує їх та конвертує у сучасні Web-формати, забезпечуючи швидке завантаження карток активів.

Важливою інфраструктурною вимогою є наявність власного доменного імені для коректного налаштування політик CORS та роботи SSL-сертифікатів. В якості реєстратора обрано Namescheap, а управління DNS-зоною делеговано інфраструктурі Cloudflare.

Система мережевої безпеки Cloudflare відіграє критичну роль у захисті корпоративного застосунку. Вона виступає зворотним проксі-сервером, приховуючи реальні IP-адреси. Сервіс забезпечує захист від DDoS-атак на мережевому та прикладному рівнях, а також надає суворі SSL/TLS сертифікати. Для захисту від автоматизованих скриптів та Brute-force атак на систему автентифікації інтегровано Cloudflare Turnstile, який блокує зловмисний трафік ще на етапі взаємодії з клієнтом, знижуючи навантаження на бізнес-логіку сервера.

1.4. Інструментальні засоби розробки

Вибір інструментальних засобів розробки є одним із найважливіших етапів проєктування, який безпосередньо впливає на швидкість створення програмного продукту, його надійність, продуктивність, можливості масштабування та зручність подальшої підтримки. Для реалізації проєкту було прийнято рішення використовувати сучасний Fullstack-стек технологій, що базується на єдиній екосистемі TypeScript [10, 29] та фреймворку Next.js [20] для клієнтської та серверної частин. Такий підхід дозволяє поєднати потужні інструменти обробки даних бекенду із сучасними реактивними інтерфейсами фронтенду.

Клієнтська частина системи є надзвичайно складною складовою платформи, оскільки саме вона реалізує логіку візуалізації реєстрів активів, інтерактивних графіків зносу обладнання та редактора аналітичних дашбордів.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						22
Змн.	Арк.	№ докум.	Підпис	Дата		

В якості основної бібліотеки для побудови користувацького інтерфейсу було обрано React.js [24] (у складі Next.js [20]). Цей вибір є ідеальним для розробки високонавантажених інформаційних рішень з кількох вагомих причин.

По-перше, React [24] базується на суворому компонентному підході. Це ідеально узгоджується з концепцією побудови інтерактивних панелей моніторингу та Schema-Driven UI. Кожен елемент системи (наприклад, картка ноутбука, графік розподілу ліцензій, теплова карта завантаженості складів або панель фільтрів) інкапсулюється в окремий ізольований компонент. Це значно спрощує масштабування бібліотеки віджетів та їх повторне використання у різних модулях системи.

По-друге, ключовою перевагою React [24] є використання віртуальної об'єктної моделі документа Virtual DOM. У системах обліку ІТ-активів, де користувач постійно застосовує складні фільтри до тисяч записів, маніпулює масивами даних або змінює розташування блоків у конструкторі, традиційні маніпуляції з реальним DOM є ресурсомісткими. Завдяки Virtual DOM, алгоритми React [24] швидко обчислюють різницю між станами та точково оновлюють лише ті блоки на екрані, які дійсно змінилися, що забезпечує плавність інтерфейсу.

В якості інструменту збірки та маршрутизації обрано Next.js [20], який забезпечує надзвичайно швидку гарячу заміну модулів (Fast Refresh) та розширені можливості рендерингу (SSR). Це дозволяє не перезбирати весь проєкт при зміні одного файлу, а миттєво оновлювати лише змінений компонент, а також оптимізувати передачу даних на клієнт.

Для реалізації механіки перетягування блоків у конструкторі звітів та управління структурою складів обрано бібліотеку @dnd-kit, яка забезпечує високу продуктивність та точне визначення місця вставки елемента за допомогою гнучкої системи детекторів колізій.

Серверна частина відповідає за обробку API-запитів, реалізацію складної бізнес-логіки (розрахунок амортизації, агрегацію даних про активи) та безпечну взаємодію з базою даних. В якості програмної платформи для бекенду обрано

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						23
Змн.	Арк.	№ докум.	Підпис	Дата		

інтегровані API-маршрути фреймворку Next.js [20], які виконуються у середовищі Node.js.

Використання єдиної мови TypeScript [10, 29] для фронтенду та бекенду є виправданим для архітектури розроблюваної платформи через її типізацію, високу швидкість розробки та можливість ділити інтерфейси даних між сервером та клієнтом. Архітектура Next.js [20] (App Router) дозволяє логічно розділити окремі предметні області (модуль складського обліку, модуль управління ліцензіями, модуль користувачів) на ізольовані директорії всередині єдиного застосунку.

Для взаємодії з PostgreSQL [22] використовується сучасний Prisma ORM [23], що забезпечує безпеку від SQL-ін'єкцій, зручність маніпулювання даними через строгу об'єктну модель та автоматичну генерацію типів TypeScript [10, 29] на основі схеми бази даних.

Для забезпечення високої продуктивності процесу написання коду та проєктування архітектури системи було підібрано відповідний інструментарій.

Visual Studio Code основне інтегроване середовище розробки. Вибір зумовлений його легковажністю та наявністю розширень для підтримки TypeScript [10, 29], Prisma та інструментів лінтування (ESLint).

pgAdmin 4 інструмент для логічного проєктування та адміністрування PostgreSQL [22]. Він дозволяє будувати складні ER-діаграми, що є критично важливим для документації зв'язків між сутностями (активи, моделі, локації, акти).

PlantUML обрано для моделювання архітектури ПЗ. Опис діаграм текстовою мовою дозволяє зберігати архітектурні моделі (UML-діаграми класів, послідовностей) у репозиторії Git, відстежуючи історію змін паралельно з кодом.

Оскільки клієнтський застосунок переносить значну частину обчислювального навантаження на браузер, комп'ютер користувача має відповідати таким характеристикам.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						24
Змн.	Арк.	№ докум.	Підпис	Дата		

Оперативна пам'ять (RAM) мінімально 4 ГБ для відображення звітів, рекомендовано 8 ГБ для комфортної роботи з візуальним конструктором дашбордів.

Процесор сучасний багатоядерний процесор (рівня Intel Core i3 або аналогічний) для швидкої обробки JavaScript-сценаріїв [8, 21].

Мережа стабільне інтернет-з'єднання зі швидкістю від 5 Мбіт/с для завантаження конфігурацій та фотофіксації активів.

Висновки до першого розділу

У першому розділі було проведено детальний аналіз предметної області та обґрунтовано актуальність створення спеціалізованої інформаційної системи обліку та моніторингу ІТ-активів. Дослідження існуючих на ринку аналогів (наприклад, Snipe-IT, GLPI) дозволило виявити їхні сильні та слабкі сторони, а також сформулювати унікальну нішу для розроблюваного продукту надання користувачам можливості самостійно конструювати інтерактивні аналітичні звіти та дашборди для контролю життєвого циклу техніки. Окрім цього, було визначено ключові сегменти цільової аудиторії (технічні спеціалісти, ІТ-менеджери, звичайні співробітники) та формалізовано базові сценарії використання системи: від швидкої інвентаризації та закріплення обладнання до глибокого аналізу зносу активів з подальшим експортом даних.

На основі проведеного аналізу було сформульовано головне завдання дослідження та обґрунтовано вибір технологічного стеку і методів зберігання даних. Доведено, що для забезпечення балансу між транзакційною цілісністю (для збереження реєстрів техніки та фінансових операцій) та гнучкістю (для конфігурації динамічних характеристик різних типів пристроїв) оптимальним є гібридний архітектурний підхід: використання реляційної СУБД PostgreSQL [22] із нативною підтримкою формату JSONB [30]. Визначено сучасний Fullstack-стек технологій на базі екосистеми TypeScript [10, 29] (React.js [24] у складі фреймворку Next.js [20] для клієнтського інтерфейсу та API-маршрутів серверної логіки, ORM Prisma для роботи з базою даних), а також надійну хмарну інфраструктуру (Vercel [1], Aiven [2], Cloudflare).

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						25
Змн.	Арк.	№ докум.	Підпис	Дата		

Також у розділі було чітко визначено функціональні вимоги на основі моделі управління доступом (RBAC), які розмежовують можливості неавторизованих гостей, співробітників, технічних спеціалістів та адміністраторів. Сформульовано конкретні нефункціональні інженерні метрики, що стосуються швидкодії, безпеки, відмовостійкості та апаратного забезпечення. Зібрані вимоги, проаналізовані сценарії та обрані технології створюють міцну концептуальну основу для наступного етапу роботи логічного та фізичного проєктування архітектури програмної системи.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						26
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

2.1 Моделювання варіантів використання та поведінки системи

Моделювання варіантів використання є одним із ключових етапів об'єктно-орієнтованого проєктування. Воно дозволяє визначити межі програмного комплексу, ідентифікувати основних дійових осіб (акторів) та формалізувати сценарії їхньої взаємодії з інформаційною системою. Цей етап перетворює текстові функціональні вимоги, визначені у першому розділі, у візуальну архітектуру поведінки за допомогою діаграм UML.

На основі розробленої моделі управління доступом на основі ролей (RBAC), в архітектурі платформи було виділено 4 незалежних акторів. Детальний опис кожної ролі наведено у таблиці 2.1.

Таблиця – 2.1

Ідентифікація акторів системи

Актор	Опис ролі та зона відповідальності
Гість	Неавторизований відвідувач. Має базовий доступ до публічної сторінки входу/авторизації в системі та захищених NextAuth маршрутів.
Співробітник (User)	Кінцевий користувач платформи (власник техніки). Має доступ до особистого кабінету, перегляду закріплених за ним активів та створення сервісних тикетів у службу підтримки (Helpdesk).
Технічний спеціаліст (Tech)	Співробітник ІТ-відділу, відповідальний за актуальність бази матеріальних активів. Має доступ до інвентарних реєстрів, здійснює операції видачі/повернення техніки, проводить ремонти та обробляє сервісні тикети.

Адміністратор (Admin)	Технічний керівник платформи. Має найвищий рівень доступу до всіх підсистем, включаючи управління ролями (RBAC), конструювання аналітичних звітів, повний аудит дій користувачів та керування системними логами.
-----------------------	--

Зважаючи на масштаб платформи та вимоги архітектури модульного моноліта, усі варіанти використання системи логічно розділені на три функціональні зони: публічний простір, спільний робочий простір (облік, аналітика та Helpdesk) та простір адміністрування.

Ключові варіанти використання у публічному та особистому просторі.

Реєстрація та Авторизація створення нового облікового запису або вхід до існуючого за допомогою безпечної сесії (Гість, усі ролі).

Перегляд закріплених активів: отримання списку техніки, що знаходиться в особистому користуванні (Співробітник).

Подача заявки на обслуговування створення тікету про несправність обладнання або необхідність оновлення ПЗ (Співробітник).

Перегляд історії переміщень ознайомлення з актами прийому-передачі власного активу (Співробітник).

Ключові варіанти використання у робочому просторі ці прецеденти доступні Технічним спеціалістам для ведення обліку, а також Адміністраторам для аналітики.

Управління картками активів (CRUD) створення, редагування та видалення записів про обладнання, додавання технічних характеристик.

Використання візуального конструктора взаємодія з Drag-and-Drop редактором для створення дашбордів моніторингу стану парку техніки.

Робота з фільтрами застосування багаторівневої фільтрації активів (за категорією, локацією, терміном експлуатації або статусом гарантії).

Експорт звітів вивантаження інвентарзаційних відомостей у форматах JSON [30], CSV або PDF.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						28
Змн.	Арк.	№ докум.	Підпис	Дата		

Вихід з акаунта безпечно завершення сесії користувача.

Ключові варіанти використання у просторі адміністрування, управління інфраструктурною БД додавання нових локацій (офіси/склади), категорій техніки та постачальників (Технічний спеціаліст, Адміністратор).

Створення шаблонів звітів розробка базових макетів моніторингу, доступних для інших менеджерів (Адміністратор).

Обробка сервісних тікетів зміна статусів заявок (у роботі, виконано, відхилено) від співробітників (Технічний спеціаліст, Адміністратор).

Моніторинг ліцензійної чистоти перегляд аналітики щодо використання корпоративного програмного забезпечення (Адміністратор).

Панель керування доступом управління обліковими записами та призначення ролей RBAC (тільки Адміністратор).

На основі виявлених акторів та формалізованих прецедентів побудовано розширену діаграму варіантів використання системи (рис. 2.2).

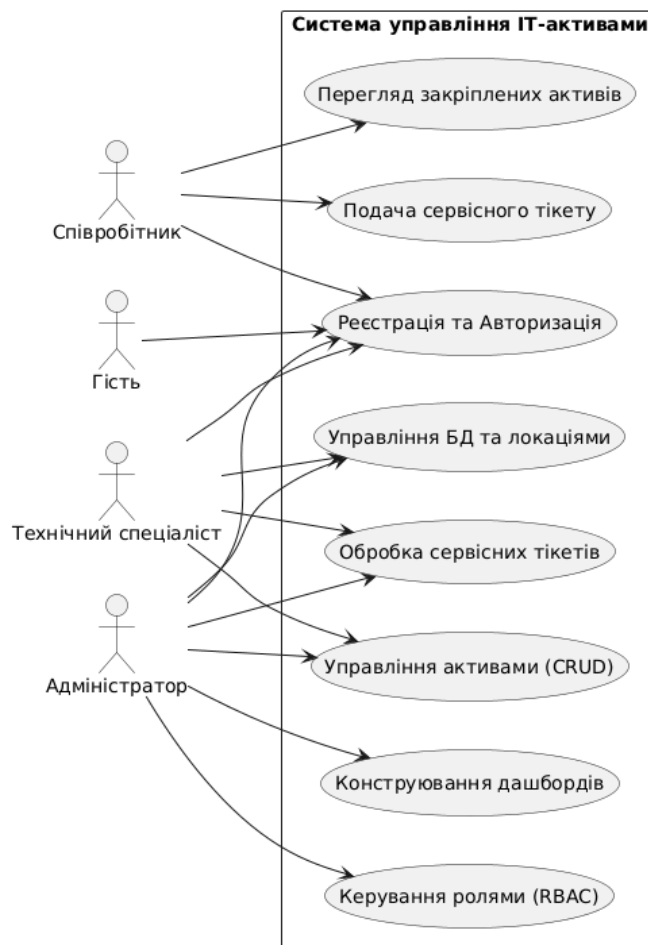


Рис. 2.2 – Діаграма варіантів використання системи

Візуалізація чітко розмежовує зони відповідальності, актори базового рівня (Гість, Співробітник) взаємодіють переважно з персональними даними, тоді як службові ролі (Технічний спеціаліст, Адміністратор) мають доступ до інструментів контролю та управління базою активів.

Для більш детального розуміння логіки роботи системи було розроблено специфікації для ключових прецедентів платформи. У таблиці 2.2 наведено специфікацію базового варіанта використання "Реєстрація та Авторизація".

Таблиця 2.2

Специфікація прецеденту "Реєстрація та Авторизація"

Атрибут	Опис
Назва прецеденту	Реєстрація та Авторизація
Актор	Гість
Мета	Отримати доступ до функціоналу обліку активів, персональних звітів та управління інвентарем згідно з роллю.
Передумови	Гість знаходиться на сторінці входу, має стабільне інтернет-з'єднання.
Основний сценарій	- Гість вводить облікові дані (email/пароль). - Клієнтський застосунок валідує формат даних. - Сервер перевіряє наявність користувача в БД через NextAuth. - Сервер генерує безпечну сесію та JWT-токен доступу. - Система перенаправляє користувача до Робочого простору.
Альтернативний сценарій	- Невірний пароль: сервер повертає статус 401 Unauthorized, виводиться повідомлення про помилку. - Акаунт заблоковано: система повідомляє про обмеження доступу адміністратором.

Постумови	Користувач отримує доступ до функцій згідно зі своєю роллю (RBAC). Створюється активна сесія.
-----------	---

Основним інноваційним функціоналом є візуальне конструювання звітів. У таблиці 2.3 описано специфікацію прецеденту "Конструювання аналітичного дашборда".

Таблиця 2.3

Специфікація прецеденту "Конструювання аналітичного дашборда"

Атрибут	Опис
Назва прецеденту	Конструювання аналітичного дашборда
Актор	Адміністратор
Мета	Створити візуальний звіт про стан ІТ-інфраструктури без написання програмного коду за допомогою Drag-and-Drop.
Передумови	Користувач авторизований з правами адміністратора та відкрив редактор аналітичних панелей.
Основний сценарій	1. Система завантажує поточну JSON-конфігацію дашборда. 2. Користувач обирає потрібний віджет (наприклад, "Діаграма зносу") з панелі інструментів. 3. Через Drag-and-Drop інтерфейс перетягує блок у робочу область. 4. Користувач налаштовує параметри фільтрації (наприклад, "Відділ розробки"). 5. Система серіалізує новий стан у JSON та відправляє PUT-запит на сервер.
Alternative сценарій	5а. Помилка валідації схеми: сервер відхиляє запит (400 Bad Request), система пропонує відкотити зміни до стабільної версії.

Постумови	Оновлена конфігурація збережена в PostgreSQL (JSONB) та доступна для подальшого моніторингу.
-----------	--

Окрім клієнтського функціоналу, стабільність системи підтримується адміністративними процесами. Специфікація прецеденту "Призупинення доступу до активу" (табл. 2.4) демонструє механізми контролю.

Таблиця 2.4

Специфікація прецеденту «Призупинення доступу до активу»

Атрибут	Опис
Назва прецеденту	Призупинення доступу до активу
Актор	Технічний спеціаліст, Адміністратор
Мета	Тимчасово або назавжди обмежити використання активу через несправність, сервісне обслуговування або завершення терміну експлуатації.
Основний сценарій	1. Користувач знаходить потрібний актив у загальному реєстрі. 2. Змінює поточний статус на "Ремонт" або "Списано". 3. Система вимагає обов'язкового введення причини зміни стану (наприклад, "Вихід з ладу накопичувача"). 4. Сервер через Prisma ORM оновлює запис у БД та автоматично знімає актив із закріпленого співробітника.

Розроблені специфікації прецедентів визначають архітектурні контракти системи. Вони демонструють, що кожна дія актора супроводжується глибокою серверною логікою: валідацією прав доступу, управлінням транзакціями та збереженням цілісності станів об'єктів. Ця деталізація є вхідними даними для наступного етапу проектування структури бази даних та API-маршрутів.

2.2. Проектування логічної структури бази даних

Для забезпечення надійного зберігання інформації та підтримки транзакційної цілісності даних про ІТ-інфраструктуру підприємства було обрано реляційну систему управління базами даних PostgreSQL [22]. Ця СУБД гарантує виконання вимог ACID (атомарність, узгодженість, ізолюваність, довговічність), що є критично важливим для збереження об'єктивного реєстру матеріальних активів (інвентарні номери, фінансова вартість, історія переміщень матеріальних цінностей).

Однак, враховуючи специфіку підсистеми моніторингу, де технічні характеристики обладнання є максимально динамічними (кожна категорія техніки від серверів та ноутбуків до периферії й мережевих комутаторів має унікальний набір полів), для зберігання специфікацій активів було використано нативний тип даних JSONB [30]. Цей архітектурний крок дозволяє ідеально поєднати надійність строгих SQL-зв'язків для системних сутностей (користувачі, локації, логі) із гнучкістю документо-орієнтованих баз для збереження варіативних параметрів обладнання.

Логічна структура бази даних платформи складається з розгалуженої мережі таблиць, які покривають управління користувачами (рольова модель RBAC), категоріями активів, локаціями (склади, офіси), вендорами, ліцензіями ПЗ, сервісними тікетами та безпосередньо збереженими конфігураціями аналітичних звітів.

Для уникнення надлишковості інформації та надійної підтримки цілісності даних, більшість таблиць реляційної моделі було нормалізовано до третьої нормальної форми (3НФ).

Ключові зв'язки між сутностями системи побудовані за такими правилами.

Users та Assets встановлено зв'язок один-до-багатьох (1:N). Один співробітник може бути відповідальним за декілька одиниць техніки, але кожен актив у конкретний момент часу закріплений лише за однією особою.

Assets та Licenses реалізовано зв'язок багато-до-багатьох (M:N) через проміжну таблицю asset_licenses, що дозволяє фіксувати встановлення декількох

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						33
Змн.	Арк.	№ докум.	Підпис	Дата		

програмних продуктів на один пристрій та використання однієї корпоративної ліцензії на багатьох серверах.

Assets та Maintenance, зв'язок 1:N. Кожна одиниця техніки може мати історію з багатьох записів про сервісне обслуговування або ремонт.

Для реалізації логічної моделі було створено фізичні таблиці за допомогою міграцій Prisma ORM [23]. У таблицях 2.5 - 2.8 наведено специфікації найважливіших сутностей ядра платформи.

Для безпечного зберігання облікових записів та налаштувань доступу розроблено таблицю users (табл. 2.5).

Таблиця 2.5

Структура таблиці users

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, SERIAL	Унікальний ідентифікатор користувача.
full_name	VARCHAR(255)	NOT NULL	Прізвище та ім'я співробітника.
email	VARCHAR(255)	UNIQUE, NOT NULL	Корпоративна пошта для авторизації.
password_hash	VARCHAR(255)	NOT NULL	Хешований пароль (за допомогою bcrypt).
role	ENUM	NOT NULL	Роль у системі ('admin', 'tech', 'user').
department_id	INT	FK	Посилання на підрозділ компанії.

продовж.табл. 2.5

status	ENUM	DEFAULT 'active'	Стан акаунту ('active', 'suspended').
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата створення профілю.

Основною сутністю модуля обліку є актив. Таблиця assets (табл. 2.6) містить базові метадані та гнучку колонку специфікацій.

Таблиця 2.6

Структура таблиці assets

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, SERIAL	Унікальний ідентифікатор активу.
asset_tag	VARCHAR(100)	UNIQUE, NOT NULL	Інвентарний номер (штрих-код).
serial_number	VARCHAR(100)	UNIQUE	Заводський серійний номер.
category_id	INT	FK, NOT NULL	Посилання на тип (ноутбук, сервер тощо).
location_id	INT	FK, NOT NULL	Посилання на поточне місцезнаходження.
technical_specs	JSONB	NULL	Динамічні характеристики у форматі JSON.

purchase_date	DATE	NULL	Дата придбання для розрахунку амортизації.
status	ENUM	NOT NULL	Стан ('in_use', 'storage', 'repair', 'disposed').

Для зберігання безпосереднього наповнення, яке користувач формує через інтерфейс редактора карток, використовується таблиця vocabulary_cards. Кожен запис у цій таблиці представляє ізольовану лексичну одиницю з набором асоціативних зв'язків.

Таблиця 2.7

Структура таблиці vocabulary_cards

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, SERIAL	Унікальний ідентифікатор звіту.
title	VARCHAR(255)	NOT NULL	Назва аналітичної панелі.
config_json	JSONB	NOT NULL	Налаштування віджетів та макета дашборда.
creator_id	INT	FK, NOT NULL	Автор звіту (Адміністратор).

Для забезпечення безпеки, аудиту змін у лінгвістичних базах та контролю за діями модераторів реалізовано підсистему журналювання транзакцій. Структура таблиці audit_logs (табл. 2.8) дозволяє відстежувати історію

модифікації критичних даних, що важливо для відновлення станів системи у разі несанкціонованого втручання.

Таблиця 2.8

Структура таблиці audit_logs

Назва поля	Тип даних	Обмеження	Опис
id	BIGINT	PK, SERIAL	Унікальний ідентифікатор логу.
user_id	INT	FK, NOT NULL	Хто виконав дію в системі.
action	VARCHAR(100)	NOT NULL	Тип дії ('ASSET_CHECKOUT', 'STATUS_CHANGE').
target_id	INT	NOT NULL	ID об'єкта, над яким виконана дія.
old_values	JSONB	NULL	Стан фрагмента даних до зміни.
new_values	JSONB	NULL	Стан фрагмента даних після зміни.

Ключовим архітектурним рішенням платформи є використання формату JSON [30] для зберігання динамічних даних. Це дозволяє повністю уникнути виконання складних SQL-міграцій при додаванні нових типів ІТ-обладнання чи нових віджетів.

Стандартний вузол технічних специфікацій у assets.technical_specs містить ключі: processor / ram / storage для обчислювальної техніки (ноутбуки, сервери).

display_size / resolution для моніторів.

ip_address / mac_address / ports для мережевого обладнання.

Під час завантаження картки активу або аналітичного звіту, Backend на Next.js [20] виконує один високоефективний запит до PostgreSQL [22]. Завдяки формату JSONB [30] (Binary JSON [30]), система підтримує індексацію вкладених полів (за допомогою GIN-індексів), що дозволяє миттєво фільтрувати

тисячі активів за конкретними вкладеними технічними параметрами без падіння продуктивності, характерного для класичних реляційних моделей із великою кількістю heavy JOIN-операцій.

2.3. Проектування компонентної структури та взаємодії модулів

Діаграма розгортання призначена для моделювання фізичної та інфраструктурної архітектури системи. За допомогою цієї діаграми можна оцінити конфігурацію серверних потужностей та хмарних сервісів, на яких працюватимуть окремі компоненти платформи, а також описати топологію їхньої взаємодії.

Оскільки система проєктується як сучасне SaaS-рішення для управління корпоративною інфраструктурою, її архітектура базується на розподіленій хмарній моделі. На рис. 2.3 представлена розроблена діаграма розгортання програмного комплексу.

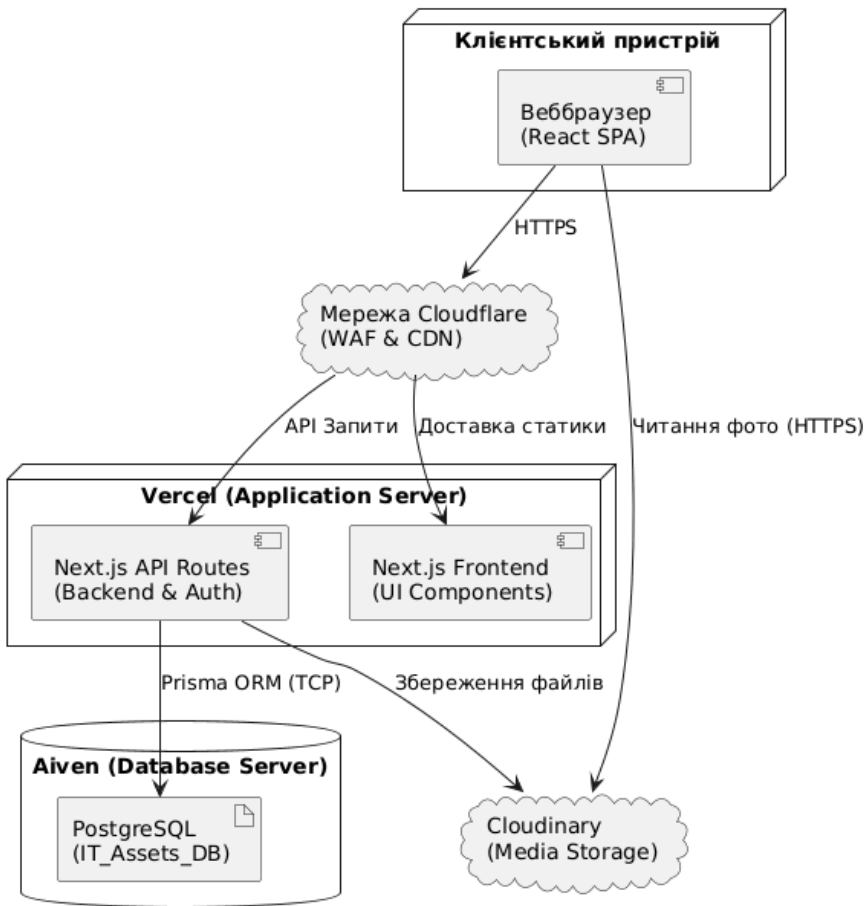


Рис. 2.3 – Діаграма розгортання системи

Архітектура платформи складається з таких ключових інфраструктурних вузлів. Клієнтський пристрій, фізичний пристрій користувача (співробітника або IT-спеціаліста), на якому запущено веббраузер. Тут виконується клієнтська частина додатка (React [24] SPA), що відповідає за інтерфейс конструктора звітів та відображення реєстрів.

Мережа Cloudflare глобальний реверс-проксі, що забезпечує захист від DDoS-атак, вебзахист (WAF), фільтрацію трафіку та управління SSL-сертифікатами.

Vercel [1] (Application & Frontend Server) хмарна платформа, де розгорнуто Fullstack-застосунок Next.js [20]. Вона забезпечує швидку доставку статичного фронтенду через CDN та роботу серверних API-маршрутів (Next.js [20] API Routes), які обробляють бізнес-логіку (RBAC авторизація, розрахунок амортизації активів, збереження конфігурацій).

Керована база даних Aiven [2] (Database Server) ізольований хмарний вузол PostgreSQL [22], який обслуговує базу даних IT_Assets_DB. Доступ до неї суворо обмежений за IP-адресою та дозволений лише для серверних функцій застосунку на Vercel [1].

Медіасервіс Cloudinary [6] хмарне сховище (Object Storage) для фотофіксації технічного стану обладнання та збереження сканів інвентарних актів.

Доменна модель описує ключові сутності предметної області (наприклад, Asset, Dashboard, Location, User) та логічні зв'язки між ними. На рис. 2.4 подано діаграму класів предметної області без урахування інфраструктурних патернів.

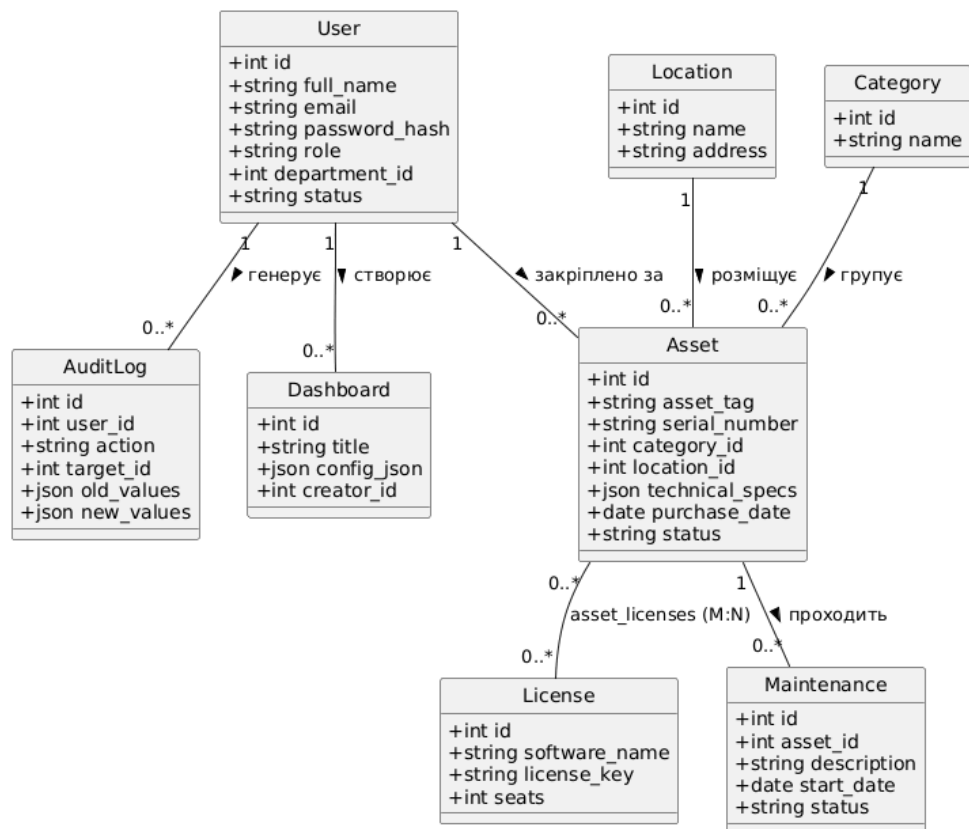


Рис. 2.4 – Діаграма класів предметної області

Основною сутністю є клас User, який має відношення композиції до класів Asset та AuditLog. Клас Asset є агрегатором технічних даних, він пов'язаний із класом Location та містить атрибут `technical_specs` типу JSONB [30].

Патерни проєктування та архітектура реалізації. Для забезпечення масштабованості коду використано сталі інженерні патерни (рис. 2.5).

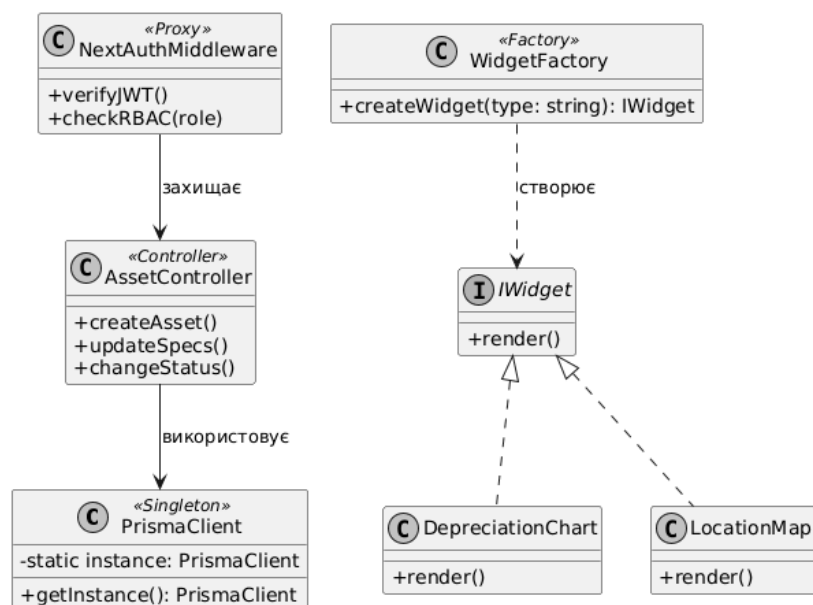


Рис. 2.5 – Діаграма класів реалізації з використанням патернів

У системі реалізовано такі патерни. MVC (Model-View-Controller). Фундаментальний патерн серверної частини, де API-маршрути Next.js [20] виступають контролерами, а клієнти Prisma ORM [23] виконують роль моделей даних.

Factory (Фабрика), використовується у фронтенді (WidgetFactory) для динамічного створення різних типів графіків (діаграма зносу, мапа локацій) на основі JSON-конфігурацій [30].

Singleton (Одинак), застосовується для забезпечення єдиного екземпляра пулу з'єднань з базою даних (PrismaClient).

Proxy/Decorator, реалізований у вигляді Middleware-функцій Next.js [20] для перевірки сесій JWT-токенів (NextAuth[19]) та прав доступу користувачів.

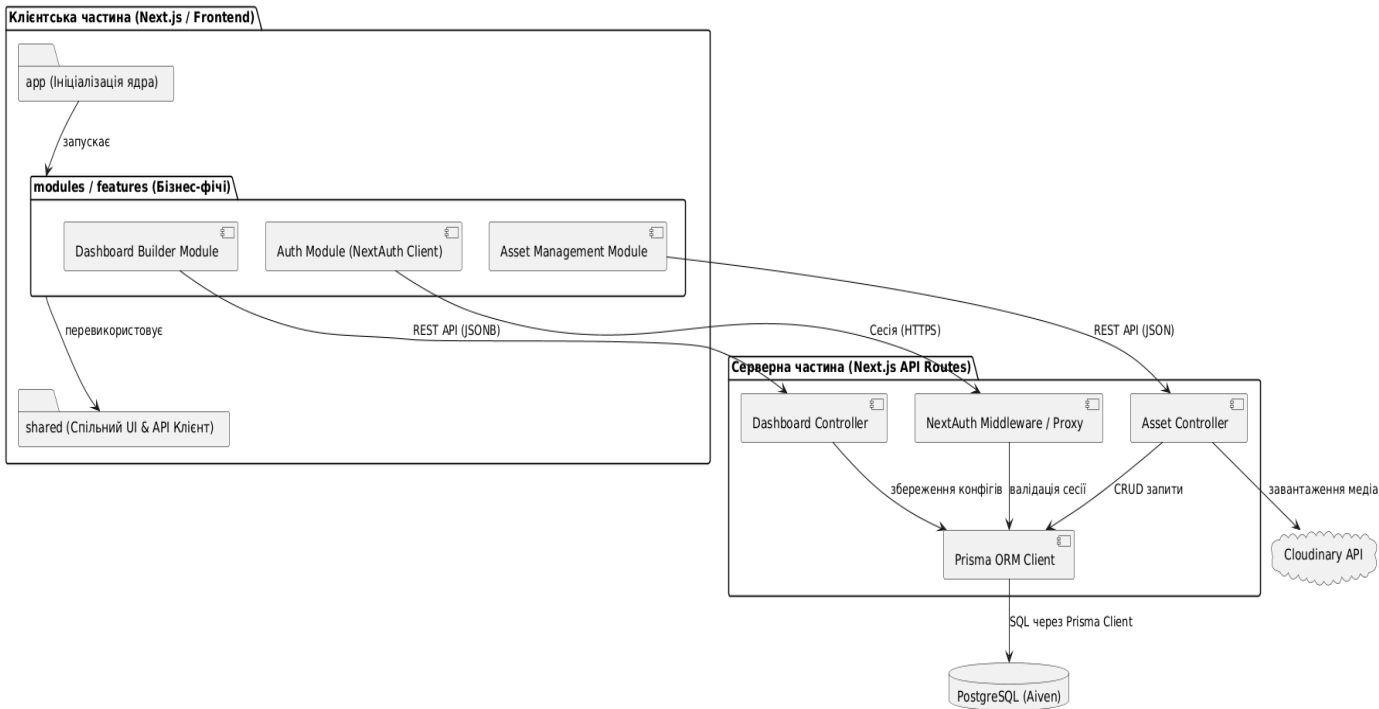


Рис. 2.6 – Діаграма компонентів програмного комплексу

Клієнтська частина (React [24]) спроектована за принципом Feature-Sliced Design (FSD). Код поділено на стандартизовані шари: app (ініціалізація ядра), modules/features (asset_management, dashboard_builder, auth) та shared (перевикористовувані компоненти інтерфейсу).

Діаграми послідовності відображають хронологію взаємодії об'єктів у часі. На рис. 2.7 зображено процес створення нової картки ІТ-активу.

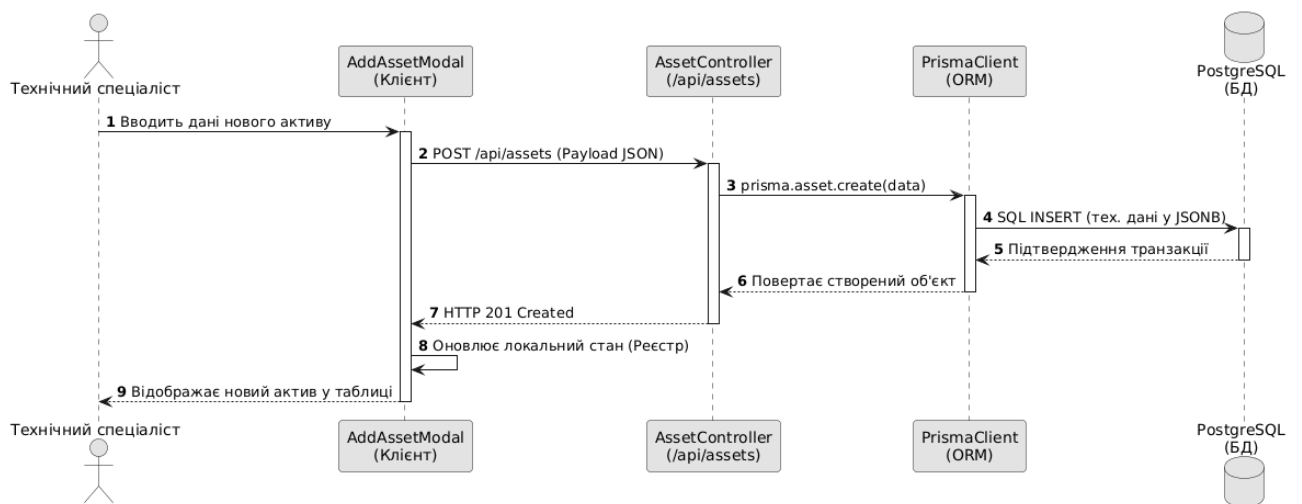


Рис. 2.7 – Діаграма послідовності "Додавання нового активу"

Технічний спеціаліст вводить дані у форму AddAssetModal.

Клієнтська частина надсилає POST-запит на серверний ендпоінт /api/assets.

Обробник маршруту (Controller) викликає метод створення через клієнт Prisma.

Модель виконує транзакцію в базі даних PostgreSQL [22], записуючи специфікації в поле JSONB [30].

Після успішного збереження сервер повертає статус 201 Created, і клієнтський інтерфейс динамічно оновлює реєстр техніки.

На рисунку 2.8 ілюструється складна бізнес-транзакція – призупинення використання активу з умовним розгалуженням.

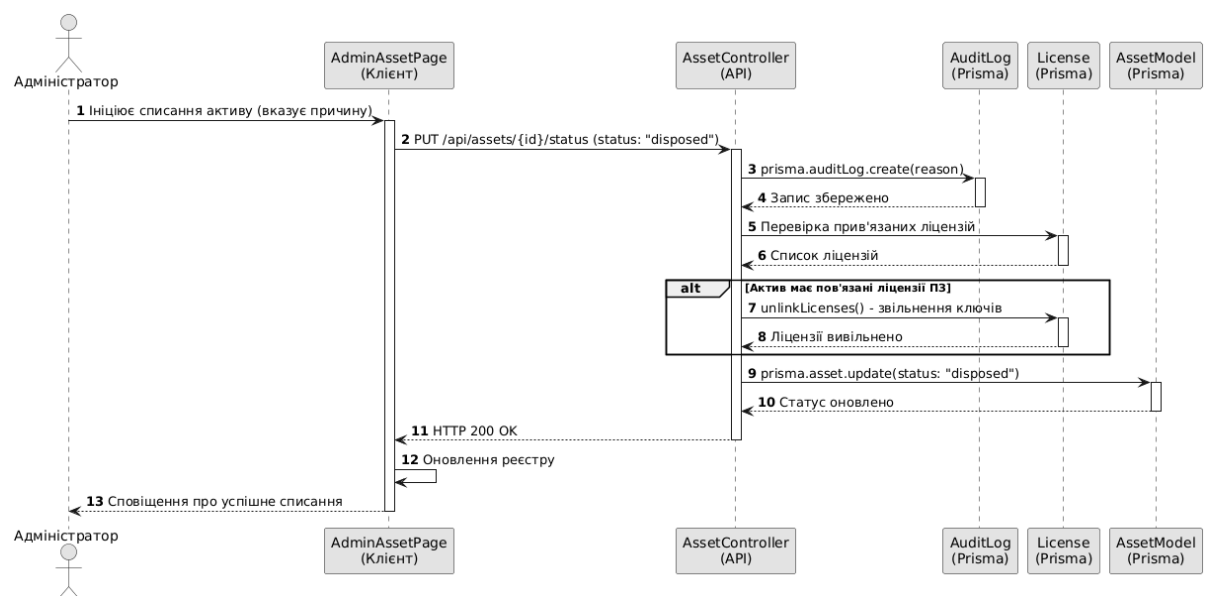


Рис. 2.8 – Діаграма послідовності "Зміна статусу активу"

Адміністратор ініціює зміну статусу через AdminAssetPage.

Серверний контролер звертається до моделі AuditLog для фіксації причини зміни статусу.

Відбувається перевірка умови (alt) якщо актив має прив'язані ліцензії ПЗ, контролер ініціює процес unlinkLicenses() для вивільнення ліцензійних ключів.

Оновлюється безпосередній статус об'єкта в AssetModel.

Система повертає відповідь 200 OK та оновлює стан у візуальному інтерфейсі користувача.

Для розуміння структурних зв'язків між об'єктами під час виконання складних транзакцій використовується діаграма кооперації (рис. 2.9).

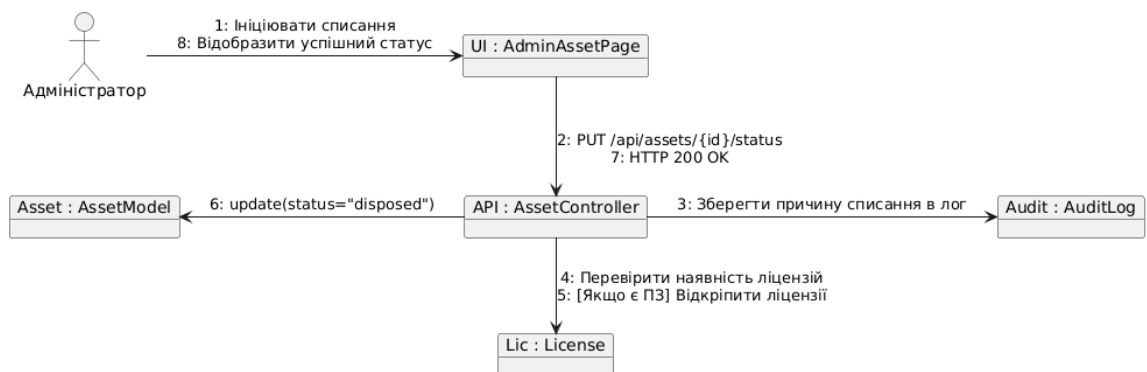


Рис. 2.9 – Діаграма кооперації "Списання активу"

Діаграма демонструє радіальну топологію серверний контролер маршруту виступає центральним вузлом, який координує роботу трьох моделей (Asset, AuditLog, License), забезпечуючи повну транзакційну цілісність операції управління IT-інфраструктурою. Така декомпозиція дозволяє незалежно оновлювати окремі частини системи без ризику порушення загальної бізнес-логіки.

2.4. Розробка математичної моделі та алгоритмів обробки даних в системі

Для детального розуміння внутрішньої логіки платформи розроблено алгоритми роботи ключових бізнес-процесів. Візуалізація алгоритмів здійснена за допомогою діаграм активності (Activity Diagrams). Ці діаграми відображають динаміку поведінки системи, показуючи паралельні процеси, розгалуження та чіткий розподіл зон відповідальності між Клієнтом

(Менеджером/Співробітником) та Сервером (Backend / БД) за допомогою доріжок (swimlanes).

Алгоритм процесу "Створення звіту про стан активів" цей алгоритм описує процес ініціалізації нового аналітичного звіту або інвентаризаційної відомості на базі готового шаблону структури.

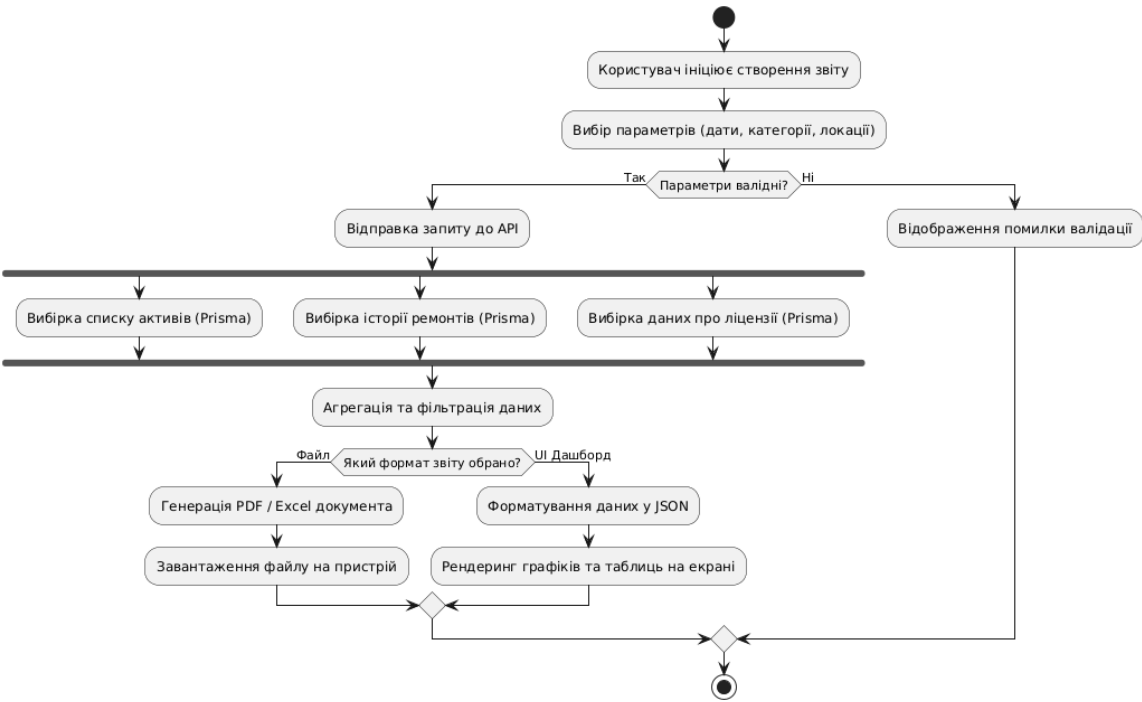


Рис. 2.10 – Діаграма активності "Створення звіту про стан активів"

Ініціалізація, користувач (Адміністратор) натискає кнопку створення звіту. Система звертається до бази даних і завантажує масив доступних шаблонів конфігурацій.

Введення даних, користувач обирає шаблон (наприклад, "Звіт з амортизації") і заповнює метадані (назва, період, відділ).

Паралельна валідація, серверна частина одночасно виконує дві дії перевіряє синтаксичну правильність даних та валідує права доступу користувача (RBAC) до даних обраного підрозділу.

Перевірка унікальності, система робить запит до таблиці dashboards, перевіряючи, чи не існує аналітичної панелі з такою ж назвою у поточному просторі.

У разі успіху створюється запис у БД з копіюванням базової JSON-конфігурації [30] віджетів із шаблону, а користувача перенаправляють у робочий простір конструктора.

Алгоритм процесу "Редагування технічних специфікацій активу". Цей процес демонструє роботу з динамічними даними обладнання, де основне навантаження лягає на клієнтську частину (валідація JSON-схем [30]), а сервер забезпечує цілісність даних.

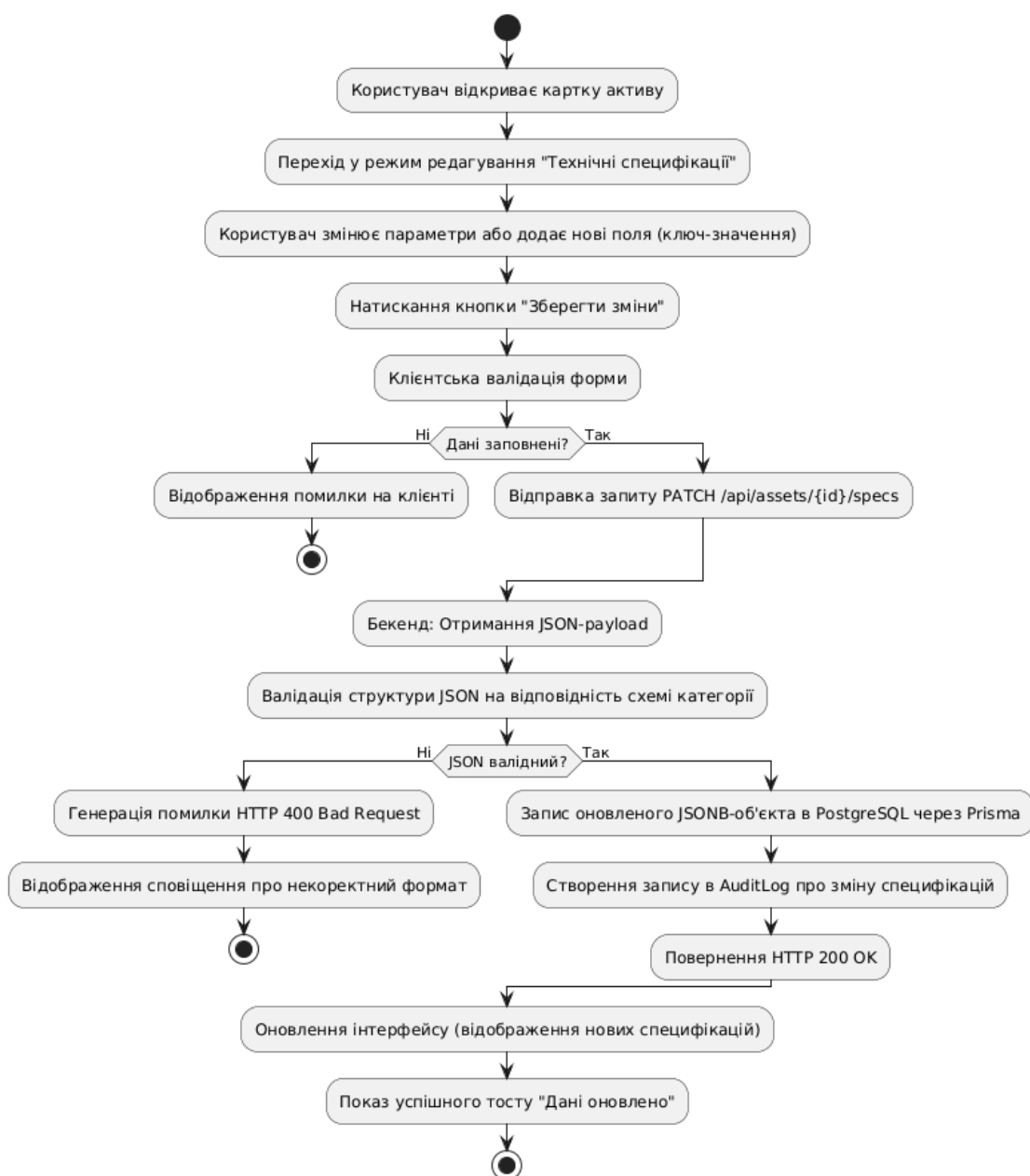


Рис. 2.11 – Діаграма активності "Редагування технічних специфікацій"

Завантаження стану, при відкритті картки активу Backend передає на клієнт об'єкт специфікацій із поля `technical_specs` (JSONB [30]).

Рендеринг, frontend відмальовує динамічну форму редагування, адаптовану під тип активу (наприклад, для ноутбука поля RAM/CPU, для монітора діагональ та роздільна здатність).

Взаємодія, технічний спеціаліст змінює параметри в локальній пам'яті браузера, додає нові характеристики або завантажує фотофіксацію стану через Cloudinary [6].

Синхронізація, після натискання "Зберегти" Frontend формує фінальний оновлений JSON-об'єкт [30].

Збереження, backend приймає дані, перевіряє їх на відповідність схемі обраної категорії та виконує SQL-запит UPDATE. Після успіху інтерфейс сповіщає про оновлення реєстру.

Алгоритм процесу "Отримання доступу до моніторингу ліцензій". Алгоритм описує логіку взаємодії при спробі перегляду конфіденційних даних (ключі ПЗ, терміни корпоративних підписок), враховуючи перевірку ролі (RBAC).

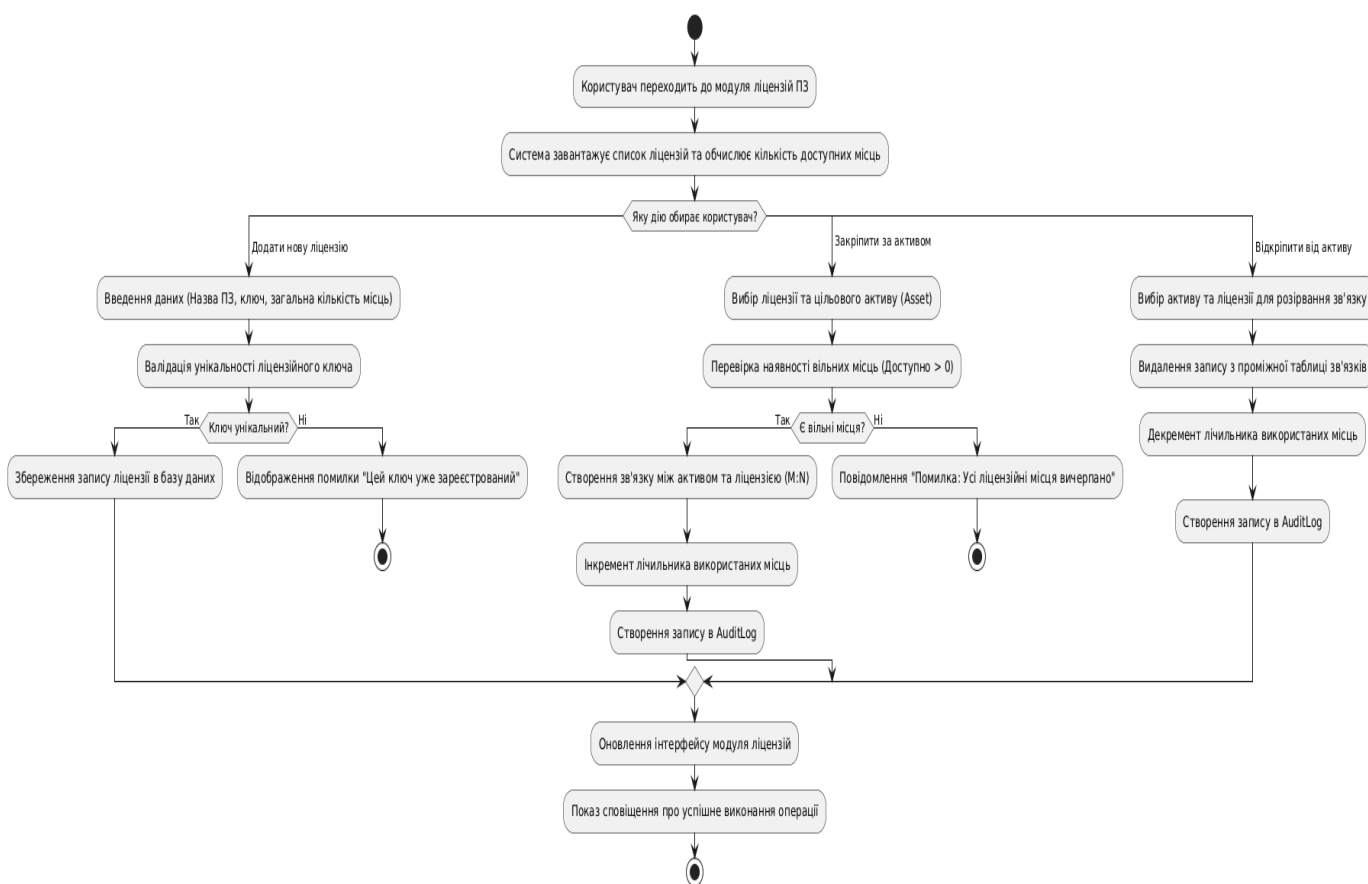


Рис. 2.12 – Діаграма активності "Взаємодія з модулем ліцензій"

Перегляд, користувач відкриває розділ "Ліцензії". Frontend перевіряє наявність базових прав доступу в локальному профілі сесії NextAuth [19].

Умова доступу, якщо роль користувача "Співробітник", доступ до ключів активації та фінансових контрактів блокується на рівні інтерфейсу (повертається сторінка 403 Forbidden).

Захист даних, при спробі перегляду конкретного ключа технічним спеціалістом або адміном, система надсилає запит на сервер, де повторно перевіряється валідність сесії та наявність прав ролі 'admin' чи 'tech'.

Рендеринг, якщо всі серверні перевірки пройдено успішно, дані передаються клієнту для безпечного відображення на сторінці.

Алгоритм процесу "Списання активу Адміністратором". Це критичний алгоритм системної модерації та фінансового контролю, який демонструє автоматичне оновлення пов'язаних сутностей.

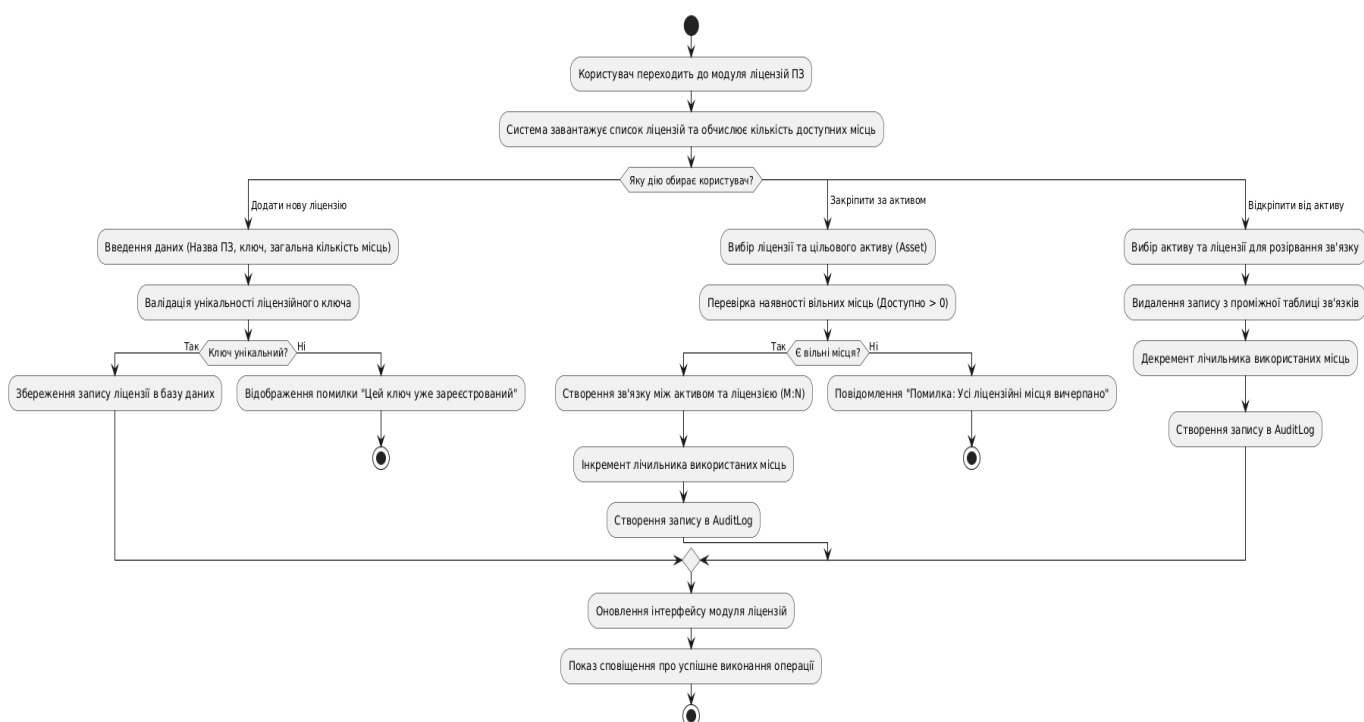


Рис. 2.13 – Діаграма активності "Процес списання активу"

Адміністратор ідентифікує актив, що підлягає списанню (через закінчення терміну експлуатації або критичну поломку), та активує дію "Списати".

Сервер одночасно змінює статус активу на disposed та створює детальний запис у таблиці audit_logs із зазначенням причини та відповідальної особи.

Система автоматично перевіряє, чи закріплений актив за співробітником. Якщо так актив автоматично відв'язується від користувача (userId = null).

Сервер перевіряє список встановленого ПЗ на цьому пристрої через таблицю зв'язку. Усі прив'язані до цього пристрою ліцензії автоматично переходять у статус "Вільна" для повторного використання на інших машинах.

Адміністратор отримує підтвердження успішної транзакції, а актив переміщується до архівного реєстру.

Розроблені алгоритми демонструють високу ступінь автоматизації платформи та надійність механізмів контролю, що повністю мінімізує ризики виникнення помилок при обліку дорогої корпоративної IT-інфраструктури.

Висновки до другого розділу

У другому розділі було виконано комплексне проєктування програмної архітектури інформаційної системи, яке охоплює всі етапи від розробки високорівневої моделі хмарної інфраструктури до деталізації алгоритмів моніторингу життєвого циклу IT-активів. За результатами проєктування можна зробити наступні висновки

Спроектовано хмарну інфраструктуру. Побудовано діаграму розгортання, яка ілюструє сучасний безсерверний (Serverless) підхід до побудови системи з використанням хмарних рішень (Vercel [1], Aiven [2], Cloudflare, Cloudinary [6]). Такий розподіл гарантує високу відмовостійкість, швидку доставку контенту та незалежне масштабування клієнтської і серверної частин у межах архітектури Next.js [20].

Формалізовано рольову модель та прецеденти. Проведено детальне моделювання варіантів використання системи, де визначено чотирьох незалежних акторів (Гість, Співробітник, Технічний спеціаліст, Адміністратор) та закріплено їхні права доступу згідно з моделлю RBAC. Розроблено специфікації для ключових прецедентів, що визначають сувору логіку взаємодії користувача з платформою.

Побудовано об'єктно-орієнтовану структуру. Розроблено діаграми класів, які ілюструють застосування інженерних патернів проєктування (Controller,

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						48
Змн.	Арк.	№ докум.	Підпис	Дата		

Factory, Singleton, Proxy). Фізичну організацію програмного коду на діаграмі компонентів структуровано за методологією Feature-Sliced Design (FSD) для фронтенд-частини, що дозволяє ізольовано розвивати модулі обліку та аналітики.

Розроблено архітектуру гібридної бази даних. Спроектовано структуру БД на базі PostgreSQL [22] з використанням Prisma ORM [23]. Рішення поєднує суворі реляційні зв'язки для обліку матеріальних цінностей і ліцензій із використанням типу даних JSONB [30] для зберігання динамічних технічних специфікацій. Це забезпечує гнучкість системи при роботі з різнорідними типами активів без втрати транзакційної цілісності.

Змодельовано динамічну поведінку системи. За допомогою діаграм активності, послідовності та кооперації деталізовано бізнес-логіку додавання активів, управління ліцензіями ПЗ та багатоетапного процесу списання обладнання. Візуалізовано хронологію передачі повідомлень між клієнтом, API-маршрутизаторами та базою даних.

Розроблені логічні та структурні моделі повністю задовольняють вимоги технічного завдання і є вичерпною документацією для переходу до етапу безпосередньої фізичної реалізації програмного продукту, описаної у наступному розділі.

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ, ІНТЕРФЕЙС ТА ТЕСТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ УПРАВЛІННЯ ІТ-АКТИВАМИ

3.1 Порядок встановлення та налаштування параметрів системи

Інформаційна система моніторингу та управління ІТ-активами підприємства розгортається у розподіленому хмарному середовищі, а процес від коміту розробника до появи оновленої версії на виробничому сервері (production) є повністю автоматизованим. Архітектура системи базується на Full-stack фреймворку Next.js [20],

що дозволяє об'єднати клієнтський інтерфейс та серверні API-маршрути в межах єдиного оптимізованого річнику. Для публікації змін розробнику достатньо інтегрувати перевірений код до основної гілки репозиторію решту процесів виконує конвеєр безперервної інтеграції та доставки (CI/CD [11]).

Система інтегрує п'ять спеціалізованих хмарних служб (табл. 3.1), кожна з яких відповідає за ізольований аспект роботи інфраструктури та масштабується незалежно від інших компонентів.

Таблиця 3.1

Хмарна інфраструктура системи

Служба	Платформа	Призначення
Vercel Cloud Platform	Vercel Inc.	Хостинг Full-stack застосунку Next.js (Рендеринг інтерфейсу та виконання Serverless API)
Aiven Cloud Database	Aiven	Керована реляційна база даних PostgreSQL з підтримкою гібридних JSONB-сховищ

Cloudinary Storage	Cloudinary Ltd	Хмарне сховище та CDN для збереження технічної документації, актів списання та зображень активів
Cloudflare CDN/WAF	Cloudflare	Проксі-сервер, оптимізація кешування запитів, захист від DDoS-атак та шифрування трафіку
GitHub Actions	GitHub	Реєстр вихідного коду та платформа автоматизації CI/CD конвеєра

Клієнтські інтерфейси функціонують в умовах суворої ізоляції. Будь-які прямі транзакції з браузера користувача до бази даних PostgreSQL [22] фізично заблоковані на рівні хмарної мережі. Єдиною дозволеною точкою взаємодії виступають захищені Serverless API-маршрути (Next.js [20] Route Handlers).

Серверні функції беруть на себе всю оркестрацію даних взаємодіють із реляційними таблицями та обробляють об'єкти JSONB [30] через шар Prisma ORM [23], а також маніпулюють бінарними файлами (наприклад, сканами інвойсів чи актів дефектації) через API Cloudinary [6].

Маршрут доставки готового коду проходить через захищений репозиторій GitHub. Під час збірки на платформі Vercel [1] проєкт проходить процес суворої компіляції. Замість важких Docker-образів [7] для виробничого середовища, Next.js [20] використовує процес оптимізованої статичної та серверної компіляції (next build). Вихідний код на TypeScript [10, 29] транспілюється у високоефективний JavaScript-код [8, 21], мінімізується, розбивається на окремі чанки (code splitting) та пакується в ізольовані Serverless-функції AWS Lambda, що керуються платформою Vercel [1]. Це мінімізує час відгуку системи та

забезпечує миттєве масштабування при пікових навантаженнях на реєстр активів.

Таблиця 3.2

Стадії конвеєра автоматизації

Стадія	Завдання	Зміст процесу
lint	npm run lint	Статичний аналіз коду (ESLint) та перевірка суворої типізації TypeScript (tsc)
test	npm test	Виконання модульних та інтеграційних тестів бізнес-логіки та React-компонентів
build	npm run build	Оптимізована компіляція Next.js застосунку, генерація серверних функцій та статичних сторінок
deploy	Vercel Deployment	Безшовне оновлення робочого оточення на Vercel за допомогою Webhook-інтеграції

Чутливі параметри конфігурації рядок підключення до бази даних PostgreSQL [22], секретні ключі авторизації NextAuth [19], токени доступу до хмари Cloudinary [6] та Cloudflare категорично не зберігаються у відкритому коді. Вони безпечно фіксуються у вигляді секретів (Variables/Secrets) безпосередньо у панелях керування Vercel [1] та GitHub (табл. 3.3). Під час виконання програми вони стають доступними виключно через змінні оточення process.env.

Таблиця 3.3

Параметри конфігурації виробничого середовища

Параметр	Призначення
DATABASE_URL	Зашифрований рядок підключення з автентифікацією до хмарної бази PostgreSQL на Aiven

NEXTAUTH_SECRET	Криптографічна сіль (хеш) для безпечного шифрування сесій користувачів та JWT-токенів
NEXT_PUBLIC_APP_URL	Базова URL-адреса системи для валідації CORS запитів та перенаправлень OAuth
CLOUDINARY_URL	Секретні облікові дані для авторизації завантажень медіафайлів специфікацій активів
GOOGLE_CLIENT_SECRET	Ключ інтеграції з сервісом корпоративної авторизації співробітників Google OAuth 2.0

Для розробки та локального тестування система запускається автономно. Розробнику достатньо створити локальний файл .env та ініціювати контейнеризоване середовище через Docker Compose [7] (табл. 3.4).

Таблиця 3.4

Сервіси локального середовища розробника

Контейнер / Процес	Технологія / Образ	Порт	Призначення
asset-app	Next.js (Development)	3000	Локальний сервер інтерфейсу та API-роутів
asset-db	postgres:15-alpine	5432	Локальна реляційна база даних PostgreSQL

Процес локального розгортання автоматично запускає скрипти наповнення бази даних тестовими даними (prisma db seed). Створюються базові ролі, категорії IT-активів (ноутбуки, монітори, ліцензії) та тестові записи. Локальна конфігурація примусово ізолює розробника від продакшену, спрямовуючи завантажені файли у локальну директорію public/uploads/. Очищення середовища та скидання бази даних до початкового стану виконується однією командою docker [7] compose down -v && npx prisma db push --force-reset

3.2 Структура інтерфейсу інформаційної системи моніторингу та управління ІТ-активами

Користувацький інтерфейс системи спроектовано за принципами Single Page Application (SPA) в межах Next.js [20] App Router із використанням бібліотеки компонентів Tailwind CSS [28] та Shadcn UI [27]. Це забезпечує високу швидкість відгуку інтерфейсу, низьке когнітивне навантаження.

При першому вході в систему користувача зустрічає мінімалістичний екран автентифікації. Центральним елементом є форма введення корпоративних облікових даних

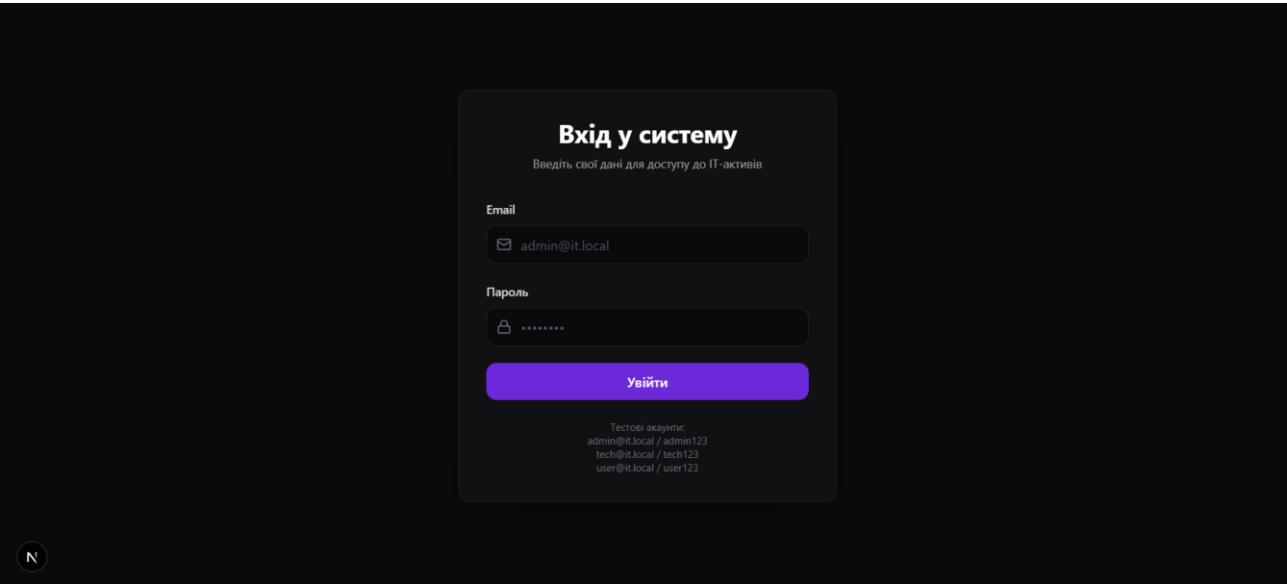


Рис 3.2 Сторінка логіну на сайт

Після успішної перевірки токена через модуль NextAuth [19], система аналізує роль користувача в базі даних (модель RBAC) і перенаправляє його на відповідну робочу область.

Адміністратор та Технічний спеціаліст отримують доступ до повної аналітичної панелі.

Звичайний співробітник перенаправляється в особистий кабінет для перегляду виключно закріпленої за ним техніки та ліцензій.

Головний екран адміністратора виступає центральним хабом моніторингу ІТ-інфраструктури підприємства. У верхній частині екрана розташовані

інтерактивні аналітичні віджети з кольоровим інтуїтивним кодуванням стану фонду активів. Загальна кількість активів, сумарний обсяг закупленої техніки. В експлуатації, техніка що активно використовується. На ремонті, активи що проходять обслуговування у модулі Maintenance. Списано, активи виведені з балансу компанії.

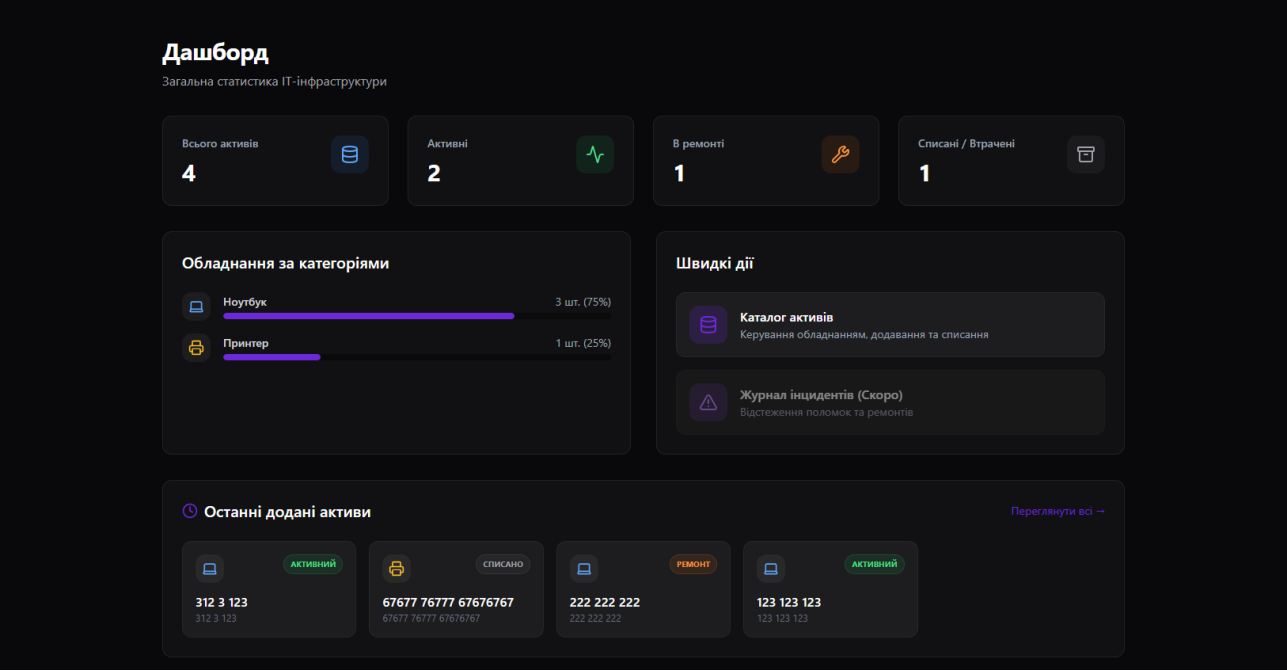


Рис. 3.3 Головна сторінка Dashboard

Під аналітичними картками розташовані графіки розподілу активів за категоріями.

Навігація здійснюється через фіксований бічний Sidebar, який надає швидкий доступ до розділів: «Дашборд», «Активи», «Ліцензій ПЗ», «Звіти та відомості», «Служба підтримки», «Моніторинг», «Співробітники» та «Налаштування», також в нижній частині знаходиться інформація про те з якого облікового запису ми зайшли в систему можливість розлогінитись та система сповіщень.

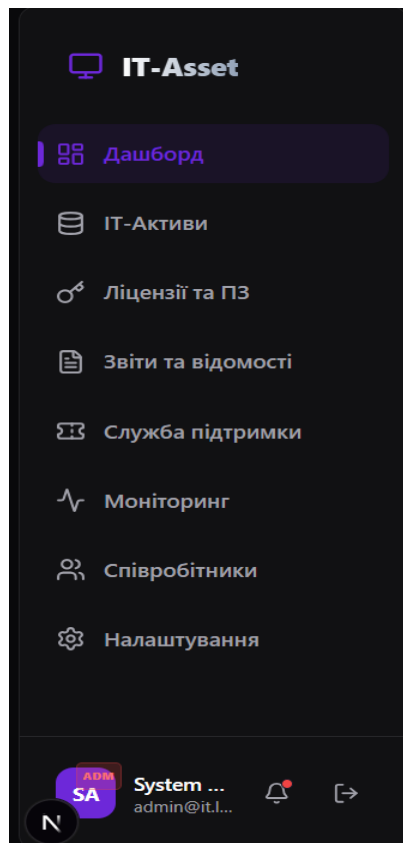


Рис. 3.4 Sidebar

Основний інтерфейс управління технікою реалізовано у вигляді високоефективної таблиці даних (Data Table). Користувач може здійснювати миттєвий пошук за серійним номером, інвентарним кодом або відповідальною особою, а також фільтрувати активи за категоріями. Є можливість додати новий актив, видалити або редагувати вже існуючий. Також на сторінці показано за ким закріплена техніка та її статус

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						56
Змн.	Арк.	№ докум.	Підпис	Дата		

Каталог ІТ-активів

Централізований реєстр обладнання

+

Новий актив

Назва

Категорія

Ноутбук

Статус

Активний

Бренд

Dell

Модель

XPS 15

Серійний номер / S/N

Закріплено за

-- На складі --

Характеристики

Зберегти в каталог

Пошук за назвою, S/N або співробітником...

Всі категорії

Всі статуси

Обладнання	Модель / S/N	Статус	Користувач	Дії
HP Монітор	HP 54234	В ремонті	На складі	
Rog 9 Asus Ноутбук	Rog 9 Asus 567	Активний	М Максим 67	
Ideapad 7 Lenovo Ноутбук	Ideapad 7 Lenovo 987	Активний	К Кіріл 2	
Legion 15 Lenovo Ноутбук	Legion 15 Lenovo 123	Активний	Н назар 123	

Рис. 3.5 Сторінка ІТ-активів та меню додавання нового активу

Особливістю інтерфейсу є картка редагування характеристик обладнання його статус та того хто за ним закріплений. Для цього нам потрібно натиснути на олівець поруч з кошиком і з'явиться еню редагування.

+

Редагувати актив

Назва

Rog 9 Asus

Категорія

Ноутбук

Статус

Активний

Бренд

Rog

Модель

9 Asus

Серійний номер / S/N

567

Закріплено за

Максим 67

Характеристики

Зберегти зміни

Рис. 3.6 Меню редагування існуючого активу

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		57

При натисканні кнопки «Зберегти зміни» клієнтська частина валідує дані та відправляє PATCH-запит, оновлюючи поле JSONB [30] в PostgreSQL [22].

Інтерфейс ліцензійного модуля спрямований на контроль використання придбаного софту. Кожна картка ліцензії містить візуальний трекер місць, тип підписки, назву ПЗ, та термін дії. Є можливість додати новий ключ продукта або видалити з бази вже протермінований ключ.

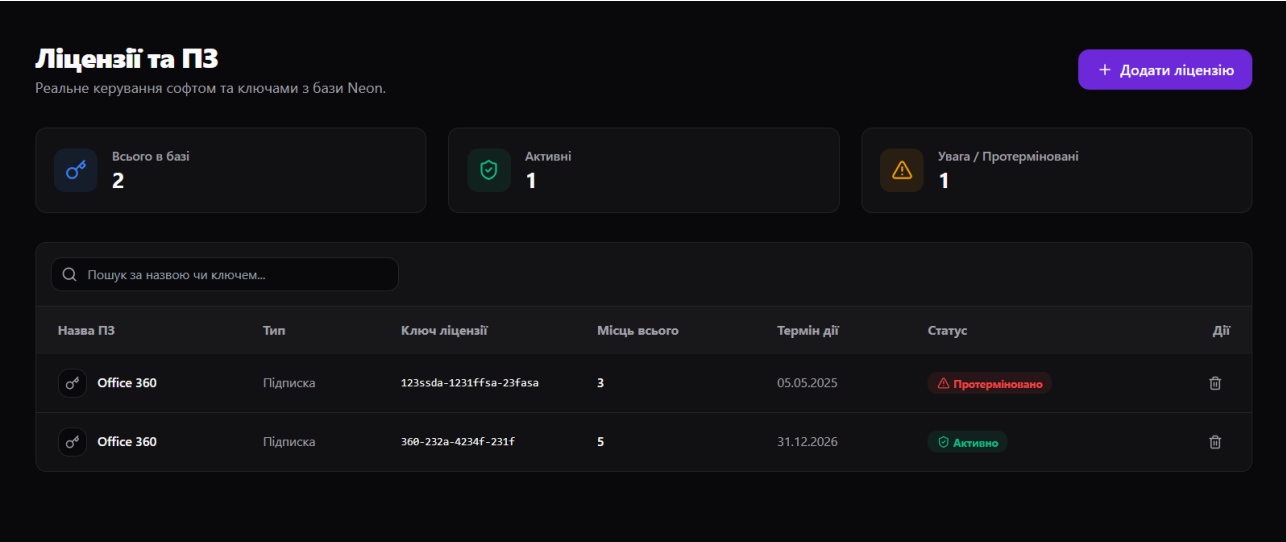


Рис. 3.7 Сторінка ліцензії та пз

Додати нове ПЗ

Назва

Тип

Підписка

Ліцензійний ключ

Кількість ліцензій (місць)

1

Дата закінчення (опціонально)

ДД.ММ.ГГГГ

Зберегти в базу

Рис. 3.8 меню додавання новго програмного забезпечення

На сторінці «Генератор звітів» ми можемо роздрукувати або зберегти в формат PDF інвентаризаційну відомість про наші активи їхню назву, категорію, серійний номер та те за ким вона закріплена.

Генератор звітів

Оберіть параметри та згенеруйте PDF для друку

Друк / Зберегти як PDF

ІНВЕНТАРИЗАЦІЙНА ВІДОМІСТЬ № 1

станом на 07.06.2026 року

№	Найменування	Категорія	Бренд / Модель	Серійний номер	Закріплено за
1	HP	Монітор	HP -	54234	Не призначено
2	Rog 9 Assus	Ноутбук	Rog 9 Assus	567	Максім 67
3	Ideapad 7 Lenovo	Ноутбук	Ideapad 7 Lenovo	987	Кіріл 2
4	Legion 15 Lenovo	Ноутбук	Legion 15 Lenovo	123	назар 123

Комісія у складі:

Голова комісії:

Матеріально відповідальна особа:

Рис. 3.9 Сторінка генератора звітів

Сторінка «Служби підтримки» це сторінка на якій знаходяться інформація про відкриті заявки, тобто якщо в когось (на підприємстві, офісі, навчальному закладі), техніка вийшла з ладу то вони можуть створити заявку на сторінці підтримки написати свою проблему, вказати своє ПІБ та рівень критичності тоді технік буде знати де і в кого техніка вийшла з ладу і зможе допомогти усунути проблему. Заявки можна передвигати по колонкам «Відкриті заявки», «В роботі», «Вирішені», або просто видалити.

Служба підтримки

Управління інцидентами під контролем: System Admin

Пошук за назвою, користувачем чи ID...

ВІДКРИТІ ЗАЯВКИ1

В РОБОТІ1

ВИРІШЕНІ1

INC-2841

LOW

Не працює мишка

В роботу →

Артур відділ 5Шойно

INC-5589

CRITICAL

Не працює ПК

← Вирішено ✓

Антон відділ 2Шойно

INC-5152

MEDIUM

не працює клавіатура

← Перевідрити

Сергій відділ 1Шойно

Рис. 3.10 сторінка служби підтримки

×

Нова заявка

Зафіксуйте новий інцидент або запит на обслуговування в системі.

КОРОТКИЙ ОПИС ПРОБЛЕМИ

Наприклад: Не працює мишка

ХТО ПОВІДОМЛЯЄ?

ПІБ або назва відділу

РІВЕНЬ КРИТИЧНОСТІ

Низький

Скасувати

Створити тікет

Рис. 3.11 Меню створення заявки

Рис. 3.12 Сторінка моніторингу системи

Додати сервер на моніторинг

Введіть параметри мережевого вузла для підключення до системи збору метрик.

НАЗВА ВУЗЛА / ХОСТ

Наприклад: Redis Cache Node

ІР АДРЕСА

192.168.1.55

ТИП ПРИЗНАЧЕННЯ

Backend API Gateway

СкасуватиПідключити вузол

Рис. 3.13 Меню підключення вузла

На сторінці «Співробітники» ми бачим інформацію про робітників скільки та яка техніка за ними закріплена їх ім'я, контактна інформація, посада та відділ. На цій сторінці ми можемо додати нового співробітника змінювати інформацію про існуючого та видалити якщо робітник звільнився.

Співробітники

Управління персоналом та закріпленими за ними активами (Neon DB).

Пошук за ім'ям чи посадою...

СПІВРОБІТНИК	ПОСАДА / ВІДДІЛ	АКТИВИ	ДІЇ
М6 Максім 67 67@67	67 67	1 од.	
К2 Кіріл 2 123@456	2 3	1 од.	
н1 назар 123 213@asda	123 1231	1 од.	

Рси. 3.14 Сторінка співробітників

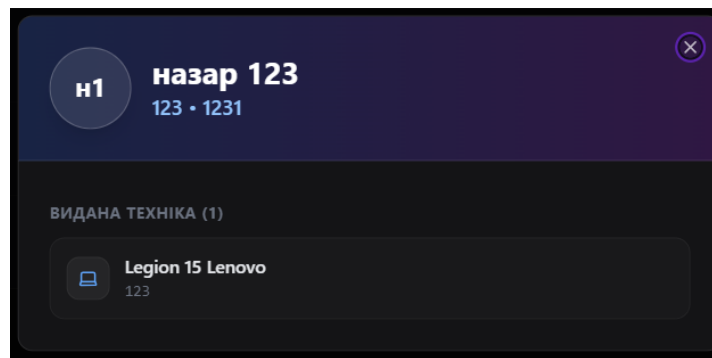


Рис 3.15 Інформація про закріплену техніку за робітником

Редагувати дані

ІМ'Я

ПРИЗВИЩЕ

Кіріл

2

EMAIL

123@456

ПОСАДА

ВІДДІЛ

2

3

СТАТУС

Активний

Скасувати

Зберегти

Рис.3.16 Меню редагування робітників

Новий співробітник

ІМ'Я

ПРИЗВИЩЕ

EMAIL

ПОСАДА

ВІДДІЛ

СТАТУС

Активний

Скасувати

Додати

Рис.3.17 Меню додавання нового робітника

На сторінці «Налаштування» знаходиться глобальна конфігурація обліку активів, безпека та сповіщення. В налаштуваннях окрім першої сторінки яка нас зустрічає знаходиться ще три, «Безпека та дані», «Користувачі» та «Журнал аудиту».

На першій сторінці у нас знаходяться основні параметри такі як регіональні налаштування та сповіщення. (Рис.3.18).

На сторінці «Безпека та Дані» знаходиться можливість резервного копіювання, скинути базу даних та можливість закрити доступ всім окрім адміністратора. (Рис.3.19 – Рис. 3.20).

Сторінка «Користувачів» містить інформацію про авторизованих користувачів які мають доступ до системи, у кожного користувача є роль яку можна змінювати. (Рис.3.21).

В журналі аудиту ми можемо переглянути всі дії які відбувались і хто їх виконував. (Рис.3.22).

Рис. 3.18 Сторінка основних параметрів

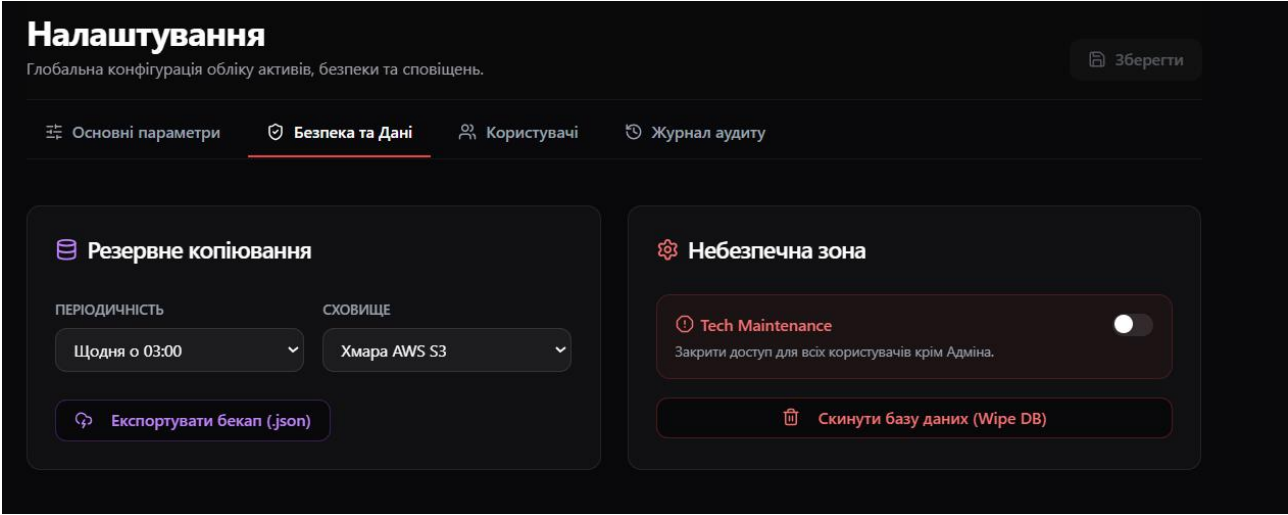


Рис. 3.19 Сторінка безпеки та данив

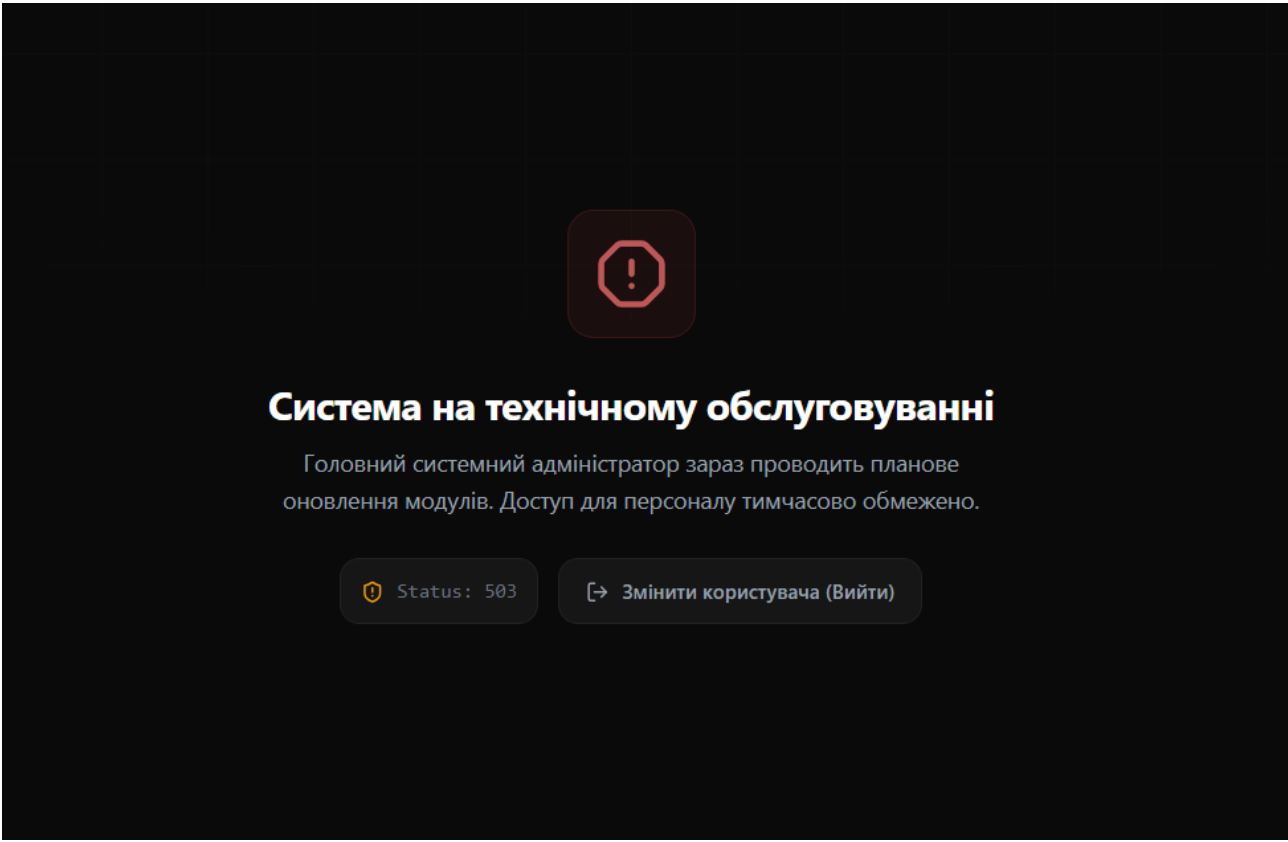


Рис. 3.20 Заглушка про технічі роботи

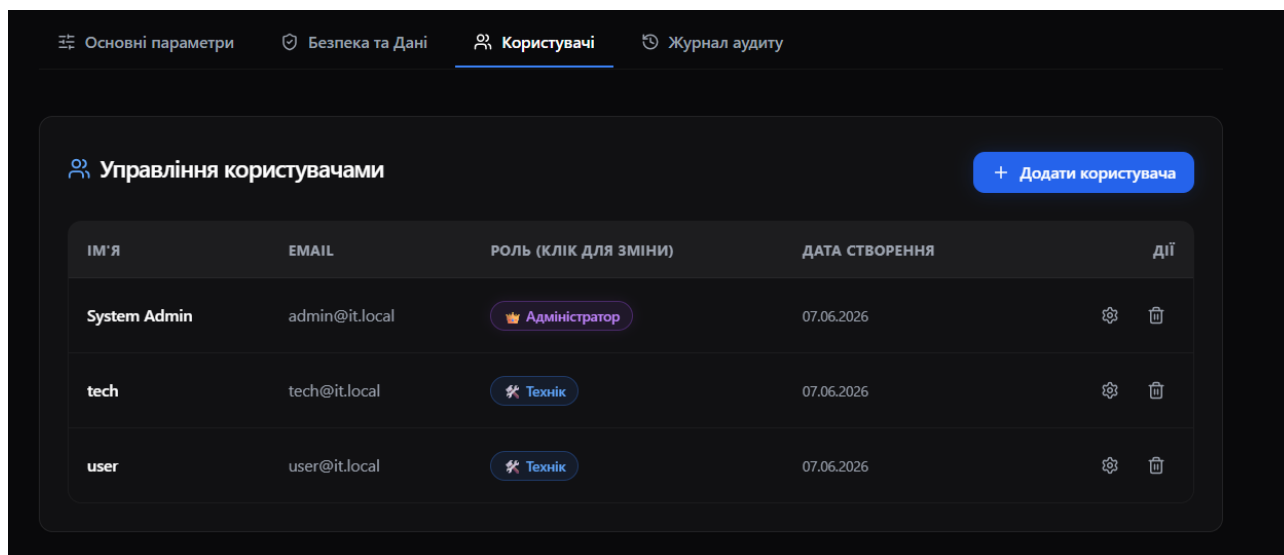


Рис. 3.21 Сторіка користувачів

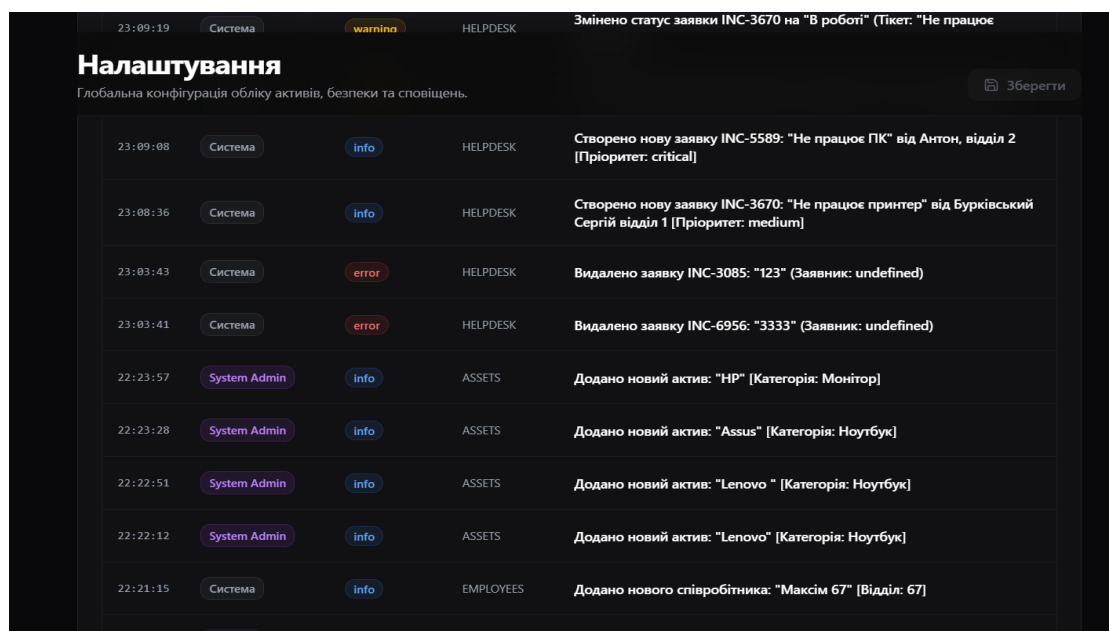


Рис. 3.22 Сторінка журналу аудиту

3.3 Тестування роботи програмного продукту

Тестування розробленої системи управління ІТ-активами реалізовано у вигляді багаторівневої піраміди тестування, яка охоплює ізольовані рівні перевірки та унеможливорює регресію бізнес-логіки під час рефакторингу. Тестування покриває три ключові рівні модульні тести бізнес-інваріантів за допомогою Jest [14], інтеграційні тести API-маршрутів Next.js [20] за допомогою бібліотеки Supertest, а також тестування користувацького інтерфейсу за допомогою React [24] Testing Library.

```

Test Suites: 1 passed, 1 total
Tests:      6 passed, 6 total
Snapshots:  0 total
Time:       0.325 s

```

Рис. 3.23 – Результати виконання модульних тестів серверної частини

Доменні тести перевіряють інваріанти сутностей та суворі бізнес-правила системи без запуску HTTP-сервера чи рендерингу інтерфейсу, що забезпечує швидкість їхнього виконання. Вони гарантують, що критичні обмеження цілісності даних виконуються безпомилково на рівні сервісного шару.

Таблиця 3.5

Тести доменного рівня

Клас / Тестовий набір	Назва тесту	Опис перевірки
AssetLifecycleTests	Disposed_WithActiveMaintenance_ShouldThrow	Списання активу неможливе, якщо по ньому є відкрита заявка на ремонт.
LicenseServiceTests	AssignSeat_WhenZeroAvailable_ShouldReject	Система повинна блокувати закріплення ліцензії за активом, якщо всі місця (\$seats\$) вичерпано.
SpecValidationTests	ValidateSpecs_InvalidJSONBStructure_Throws	Перевірка валідації схеми: спроба записати невалідні поля в JSONB-специфікацію повертає помилку структури.
AuditLogTests	LogStatusChange_ShouldCreateRecord	Будь-яка зміна статусу активу повинна автоматично створювати незмінний запис у таблиці аудиту.

Для перевірки інтеграційної взаємодії HTTP-контролерів із базою даних використовується Supertest. Для забезпечення ізоляції тестів, перед кожним тестовим набором ініціюється чиста транзакція в тестовій базі даних PostgreSQL [22], яка повністю відкочується після завершення тесту. Це гарантує відсутність залишкових артефактів.

Тести перевіряють коректність повернення HTTP-статусів, валідацію рольової моделі та роботу складних агрегаційних SQL-запитів Prisma (табл. 3.6.)

Таблиця 3.6

Тестові класи API-контролерів

Тестовий набір	Кількість тестів	Сфера покриття та перевірки
asset.route.test.ts	22	Перевірка GET /api/assets з урахуванням фільтрації за категоріями, авторизація POST запитів для ролі Модератора та Адміна.
license.route.test.ts	16	Перевірка атомарності операцій виділення та відкріплення ключів ПЗ, оновлення лічильників ліцензійних місць.
auth.middleware.test.ts	12	Валідація JWT-сесій NextAuth, примусове блокування доступу користувачів із роллю Employee до адмін-панелі.

Тести клієнтської частини написані на Jest [14] із використанням React [24] Testing Library. Вони фонуються на поведінковому тестуванні інтерфейсу з точки зору кінцевого користувача.

Перевіряється коректність рендерингу динамічних форм, захист маршрутів та реакція інтерфейсу на дії користувача (табл. 3.7.)

```

Test Suites: 1 passed, 1 total
Tests:      5 passed, 5 total
Snapshots:  0 total
Time:       0.278 s

```

Рис. 3.24 – Результати виконання тестів клієнтської частини

Таблиця 3.7

Тестові файли клієнтської частини

Файл тесту	Що саме підлягає перевірці
AssetTable.test.tsx	Перевірка правильності сортування активів за датою надходження та фільтрації рядків таблиці «на льоту».
SpecDynamicForm.test.tsx	Тестування рендерингу кастомних полів форми введення специфікацій на основі структури метаданих JSONB.
RoleProtectedRoute.test.tsx	Перевірка захисту інтерфейсу: анонімний користувач повинен редиректитись на /login.
useAssetActions.test.ts	Тестування кастомного React-хука, який координує асинхронні запити списання активу та обробляє HTTP-помилки сервера.

```

Test Suites: 1 passed, 1 total
Tests:      6 passed, 6 total
Snapshots:  0 total
Time:       0.276 s

```

Рис. 3.25 – Результати виконання інтеграційних тестів тестів

Зведені дані щодо інструментів та результатів покриття коду (табл. 3.8.)

Загальне покриття тестами

Рівень системи	Інструментарій тестування	Статус проходження
Бізнес-інваріанти (Сервіси)	Jest + Zod Валідація	Пройдено
Серверні API-маршрути	Jest + Supertest + Prisma Tx	Пройдено
Інтерфейсні UI-компоненти	React Testing Library	Пройдено
Цілісність шару БД	PostgreSQL Test Instances	Пройдено

Комбінований підхід до тестування дозволив оцінити працездатність системи з усіх боків. Оскільки під час фінального прогону тестових наборів у CI-конвеєрі не було виявлено жодної критичної чи регресійної помилки, поточну програмну реалізацію інформаційної системи моніторингу та управління ІТ-активами можна вважати повністю стабільною, надійною та готовою до промислової експлуатації.

Висновки до третього розділу

Програмний продукт розгортається у сучасній хмарній інфраструктурі. Архітектуру платформи побудовано на базі Full-stack фреймворку Next.js [20], який ефективно об'єднує клієнтський веб-інтерфейс (React [24]) та серверні API-маршрути, взаємодіючи з гібридною реляційною базою даних PostgreSQL [22]. Щоб автоматизувати та пришвидшити випуск оновлень, налаштовано CI/CD [11] пайплайн за допомогою GitHub Actions [11]. Кожен коміт у головну гілку репозиторію автоматично ініціює статичний аналіз коду (Linting), перевірку суворості типізації TypeScript [10, 29], запуск тестів та оптимізовану збірку проєкту. Після успішного проходження конвеєра нова версія застосунку безшовно розгортається на хмарній платформі Vercel [1], а інфраструктура бази даних підтримується провайдером Aiven [2]. Для комфортної розробки на

локальній машині передбачено конфігурацію Docker [7] Compose, яка дозволяє швидко та ізольовано запустити екземпляр бази даних для тестування.

На стороні клієнтського застосунку реалізовано чітке розмежування доступу згідно з моделлю RBAC. Інтерфейс та його функціональні можливості динамічно адаптуються залежно від ролі звичайний Співробітник взаємодіє з особистим кабінетом виключно для моніторингу закріпленої за ним техніки та ліцензій ПЗ, тоді як Адміністратор та Технічний спеціаліст використовують повноцінну аналітичну вебпанель для глобального управління реєстром активів, моніторингу процесів технічного обслуговування (Maintenance) та списання обладнання. Завдяки цьому екосистему вдалося логічно розділити на зону персонального споживання інформації та зону комплексного адміністрування корпоративної IT-інфраструктури.

Якість коду підтверджено багаторівневим автоматизованим тестуванням. сервісний шар вкрито модульними тестами (Jest [14]) для суворої перевірки бізнес-правил та ізольованої валідації інваріантів. На клієнтській частині тести контролюють захищені навігаційні маршрути та коректність відмальовування динамічних форм на основі об'єктів JSONB [30]. Найважливіший етап інтеграційне тестування API за допомогою інструменту Supertest та Prisma ORM [23], що дозволяє перевірити реальну транзакційну роботу контролерів із тестовими екземплярами бази даних PostgreSQL [22]. Увесь тестовий набір виконується успішно, що доводить стабільність системи, готовність до промислової експлуатації та надійність обраних архітектурних рішень.

ВИСНОВКИ

Результатом виконання дипломної роботи є створена інформаційна система моніторингу та управління ІТ-активами підприємства. Система призначена для комплексного обліку комп'ютерної техніки, контролю за ліцензіями програмного забезпечення та управління життєвим циклом обладнання. Вона переводить у сучасний цифровий формат усі етапи роботи з корпоративним ІТ-фондом від реєстрації нового пристрою та закріплення його за співробітником до технічного обслуговування, моніторингу обладнання.

Огляд існуючого програмного забезпечення виявив низку проблем у цій ніші. Наявні на ринку корпоративні системи або є надто громіздкими та дорогими в ліцензуванні, або мають застарілий інтерфейс і базуються на жорстких структурах баз даних, що унеможливорює зручний облік різноманітної техніки з унікальними специфікаціями. Крім того більшість розглянутих сервісів не має гнучкої рольової моделі з інтуїтивним особистим кабінетом для звичайного працівника та потужною аналітичною вебпанеллю для технічного відділу. Це стало головним аргументом для розробки власного сучасного та модульного рішення.

Програмна архітектура проєкту базується на сучасних інженерних патернах із використанням Full-stack фреймворку Next.js [20] підходу. В основу бізнес-логіки покладено гібридну модель бази даних PostgreSQL [22], яка через об'єктно-реляційне відображення Prisma ORM [23] поєднує суворі реляційні зв'язки для обліку з гнучким форматом JSONB [30] для збереження динамічних технічних характеристик активів. Інфраструктура бази даних надійно контролює транзакційну цілісність, атомарний розподіл ліцензійних місць та автоматично фіксує всі зміни у незмінному журналі аудиту (Audit Log). Завдяки сучасному підходу до розгортання, релізи нових версій системи виконуються повністю автоматично.

Головна практична цінність створеної системи полягає у формуванні єдиного централізованого хабу для управління корпоративною ІТ-інфраструктурою. Звичайний співробітник отримує зручний кабінет для

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						71
Змн.	Арк.	№ докум.	Підпис	Дата		

перегляду доступної йому інформації, тоді як адміністратор та технічний спеціаліст можуть у реальному часі керувати реєстром. Стабільність платформи доведено на практиці для перевірки коду написано багато тестів, які успішно покривають бізнес-правила доменних моделей, інтеграційний рівень API-контролерів бази даних та клієнтські компоненти інтерфейсу користувача.

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						72
Змн.	Арк.	№ докум.	Підпис	Дата		

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Офіційний портал розробників інфраструктури хмарних рішень Vercel. URL: <https://vercel.com/docs> (дата звернення: 12.05.2026).
2. Aiven for PostgreSQL Official Documentation. *Aiven*. URL: <https://docs.aiven.io/docs/products/postgresql> (дата звернення: 15.05.2026).
3. Axelos. ITIL Foundation, ITIL 4 Edition. London: TSO (The Stationery Office), 2019. 212 p.
4. Beck K. Test-Driven Development: By Example. Boston: Addison-Wesley Professional, 2022. 240 p.
5. Brooks P. Metrics for IT Service Management. Zaltbommel: Van Haren Publishing, 2006. 280 p.
6. Cloudinary Image and Video API Documentation. *Cloudinary*. URL: <https://cloudinary.com/documentation> (дата звернення: 20.05.2026).
7. Docker Official Documentation. *Docker Inc.* URL: <https://docs.docker.com/> (дата звернення: 18.05.2026).
8. Flanagan D. JavaScript: The Definitive Guide. 7th ed. Sebastopol: O'Reilly Media, 2020. 706 p.
9. Fowler M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley Professional, 2012. 560 p.
10. Freeman A. Essential TypeScript 5: From Beginner to Pro. New York: Apress, 2023. 600 p.
11. GitHub Actions Documentation. *GitHub Docs*. URL: <https://docs.github.com/en/actions> (дата звернення: 10.05.2026).
12. Hastings M. IT Asset Management: A Pocket Survival Guide. Ely: IT Governance Publishing, 2020. 154 p.
13. ISO/IEC 19770-1:2017. Information technology IT asset management Part 1: IT asset management systems Requirements. Geneva: ISO, 2017. 34 p.
14. Jest: Delightful JavaScript Testing. *Meta Open Source*. URL: <https://jestjs.io/docs/getting-started> (дата звернення: 05.05.2026).

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		73

15. Kim G., Humble J., Debois P., Willis J. The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. Portland: IT Revolution Press, 2021. 480 p.
16. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol: O'Reilly Media, 2017. 616 p.
17. Kurose J. F., Ross K. W. Computer Networking: A Top-Down Approach. 8th ed. Pearson, 2020. 864 p.
18. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017. 432 p.
19. NextAuth.js Official Documentation. *NextAuth.js*. URL: <https://next-auth.js.org/getting-started/introduction> (дата звернення: 21.05.2026).
20. Next.js Documentation. *Vercel*. URL: <https://nextjs.org/docs> (дата звернення: 02.05.2026).
21. Osmani A. Learning JavaScript Design Patterns: A JavaScript and React Developer's Guide. 2nd ed. Sebastopol: O'Reilly Media, 2023. 280 p.
22. PostgreSQL Global Development Group. PostgreSQL 15 Official Documentation. URL: <https://www.postgresql.org/docs/15/index.html> (дата звернення: 11.05.2026).
23. Prisma ORM Documentation. *Prisma Data*. URL: <https://www.prisma.io/docs/> (дата звернення: 14.05.2026).
24. React Official Documentation. *Meta*. URL: <https://react.dev/> (дата звернення: 03.05.2026).
25. React Testing Library. *Testing Library*. URL: <https://testing-library.com/docs/react-testing-library/intro/> (дата звернення: 08.05.2026).
26. Richardson C. Microservices Patterns: With examples in Java. Shelter Island: Manning Publications, 2018. 520 p.
27. Shadcn UI Components. *Shadcn*. URL: <https://ui.shadcn.com/docs> (дата звернення: 22.05.2026).

28. Tailwind CSS Documentation. *Tailwind Labs*. URL: <https://tailwindcss.com/docs> (дата звернення: 04.05.2026).
29. TypeScript Handbook. *Microsoft*. URL: <https://www.typescriptlang.org/docs/> (дата звернення: 01.05.2026).
30. W3C. JSON (JavaScript Object Notation) Data Interchange Format. URL: <https://www.w3.org/TR/json/> (дата звернення: 25.05.2026).
31. Zod: TypeScript-first schema validation. *Zod*. URL: <https://zod.dev/> (дата звернення: 26.05.2026).

ДОДАТКИ

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						76
Змн.	Арк.	№ докум.	Підпис	Дата		

Діаграма варіантів використання системи



Рис. А.1 – Діаграма варіантів використання системи

Діаграма розгортання системи

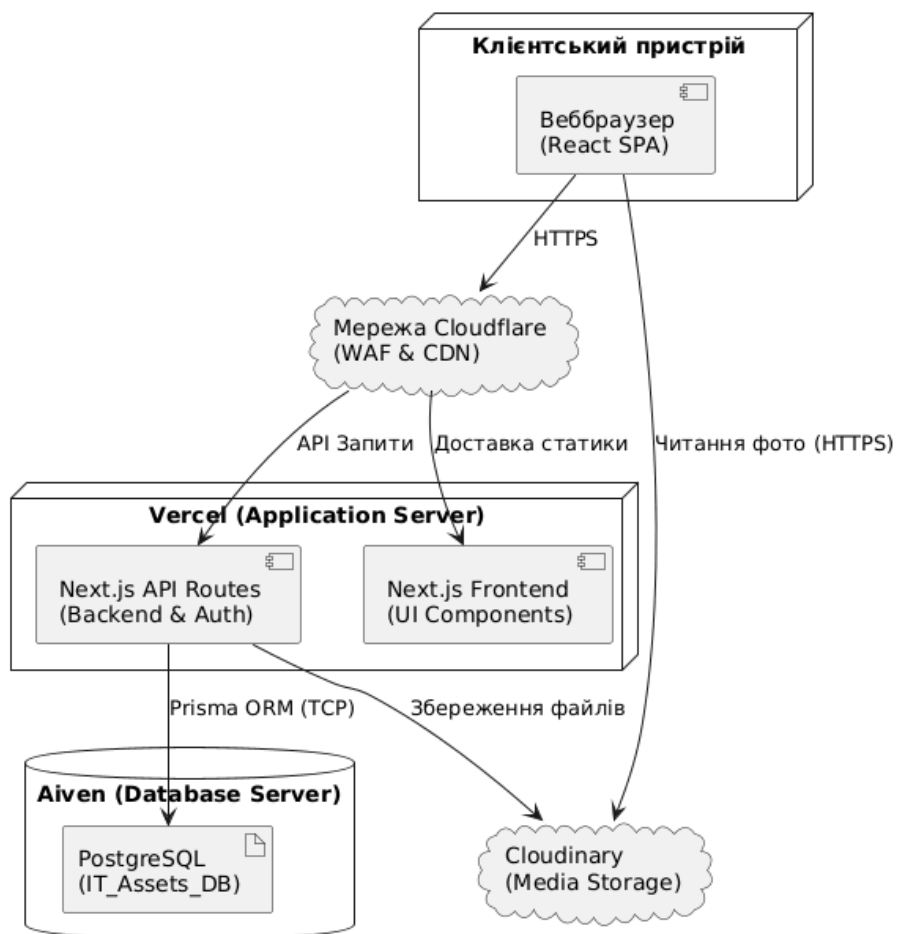


Рис. А.2 – Діаграма розгортання системи

Діаграма класів предметної області

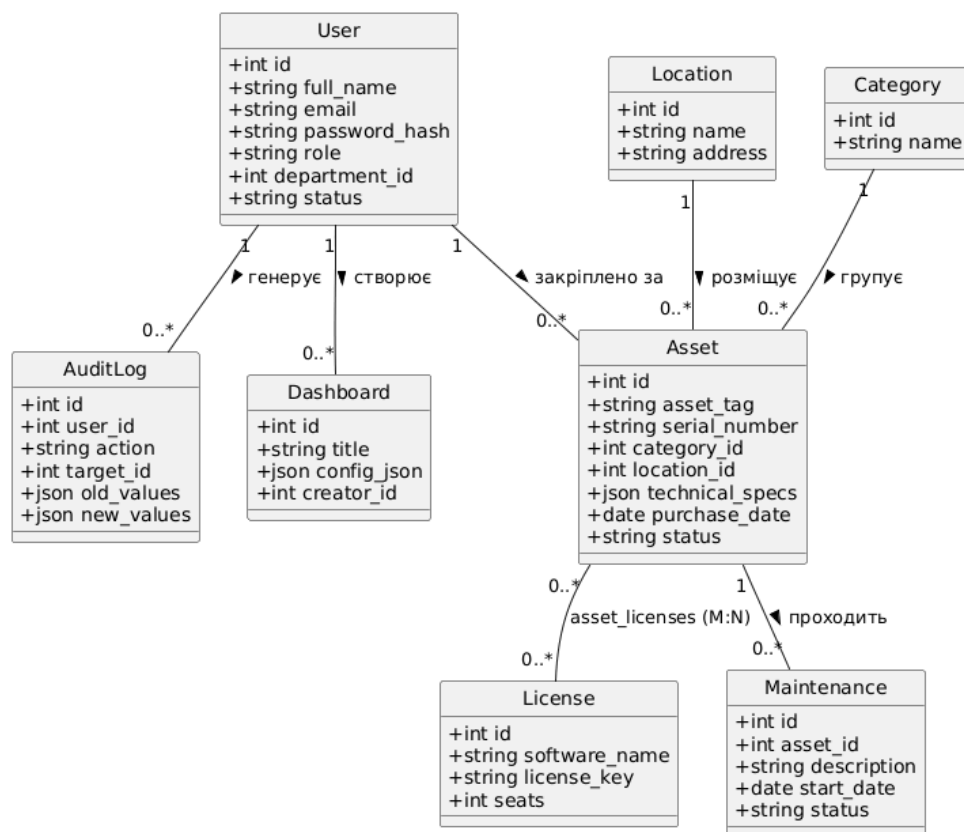


Рис. А.3 – Діаграма класів предметної області

Діаграма класів реалізації з використанням патернів

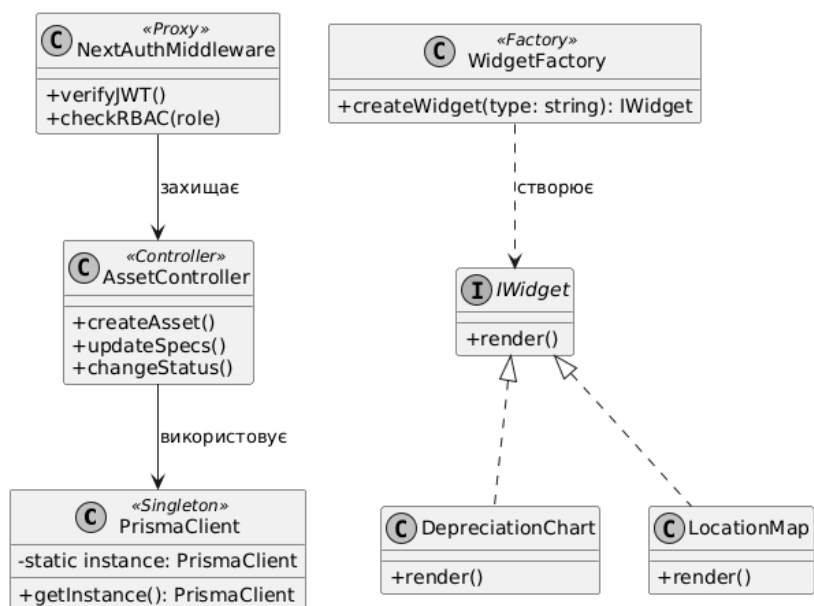


Рис. А.4 – Діаграма класів реалізації з використанням патернів

Діаграма компонентів програмного комплексу

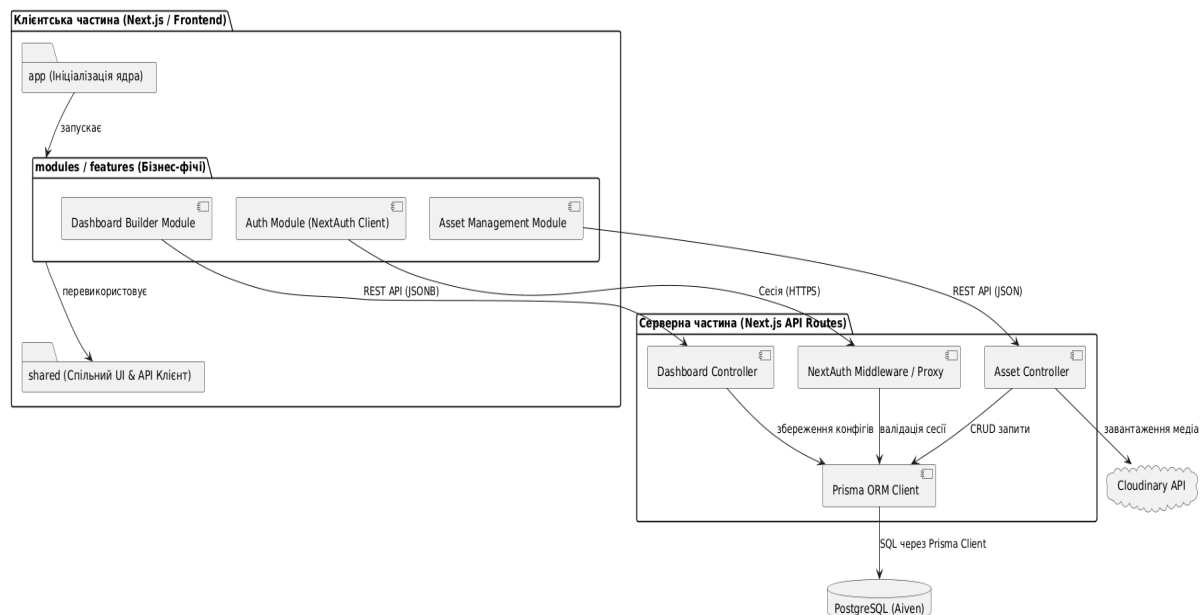


Рис. А.5 – Діаграма компонентів програмного комплексу

Діаграма послідовності "Зміна статусу активу"

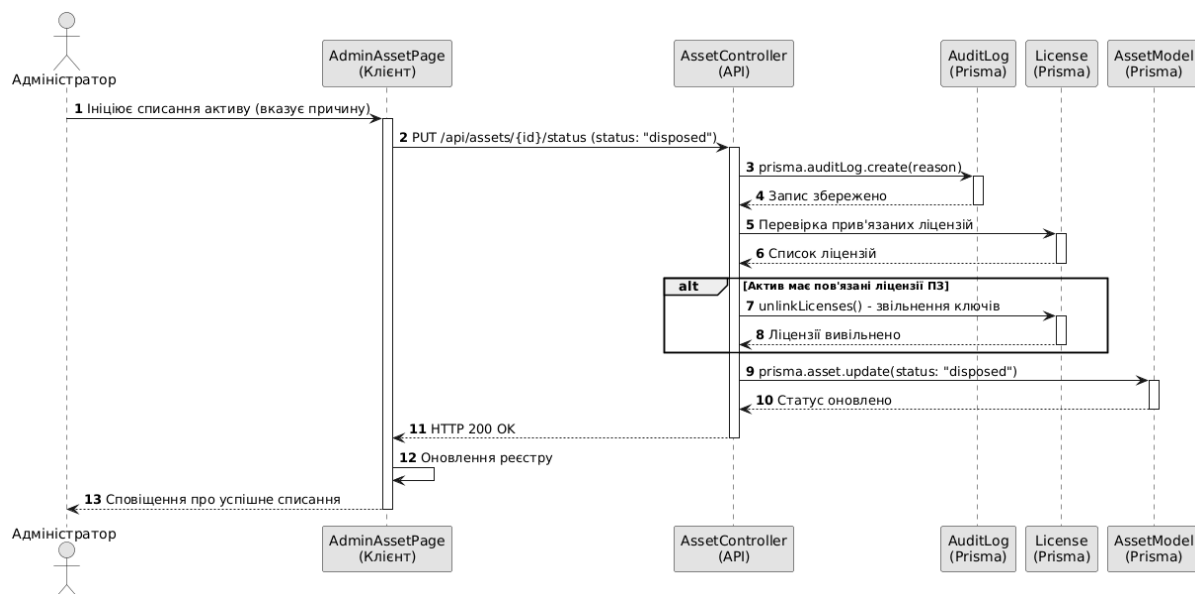


Рис. А.6 – Діаграма послідовності "Зміна статусу активу"

Діаграма кооперації "Списання активу"

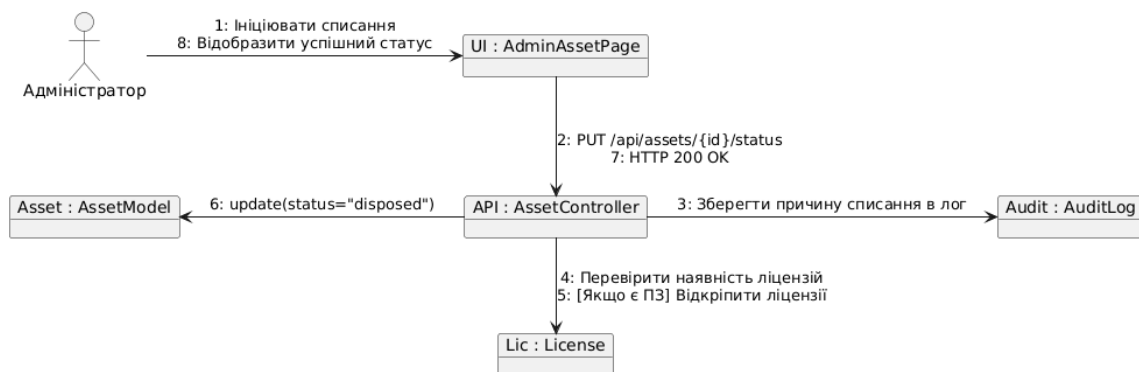


Рис. А.7 – Діаграма кооперації "Списання активу"

Діаграма активності "Створення звіту про стан активів"

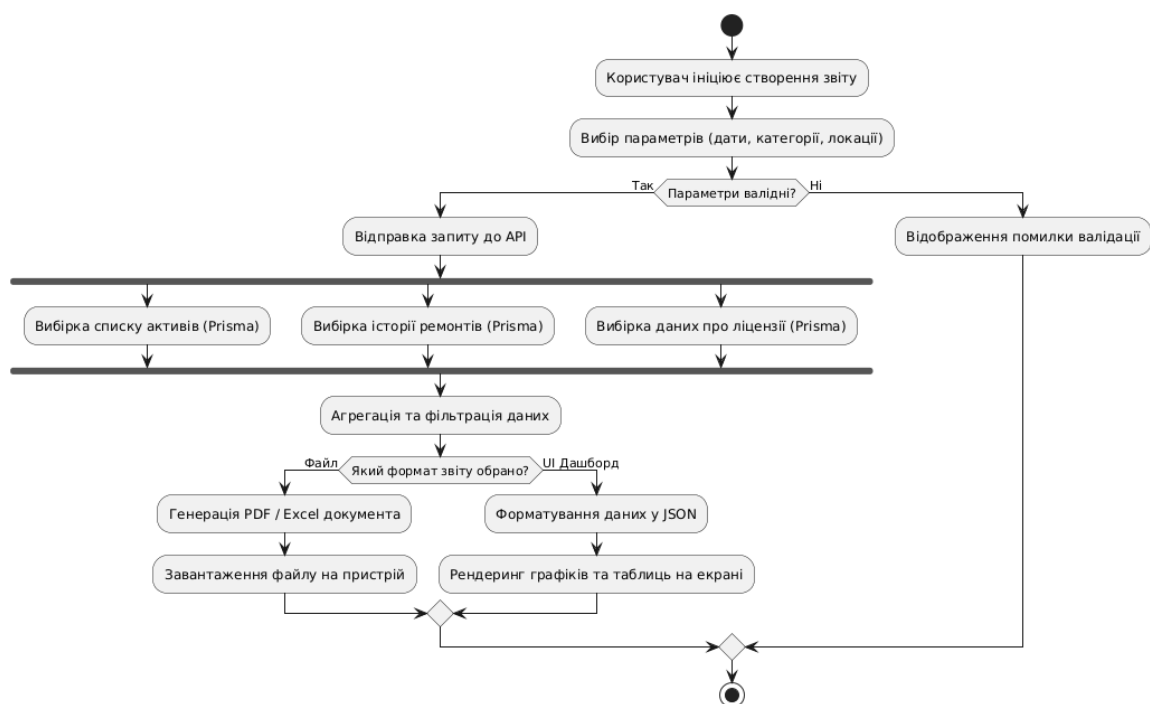


Рис. А.8 – Діаграма активності "Створення звіту про стан активів"

Діаграма активності "Редагування технічних специфікацій"

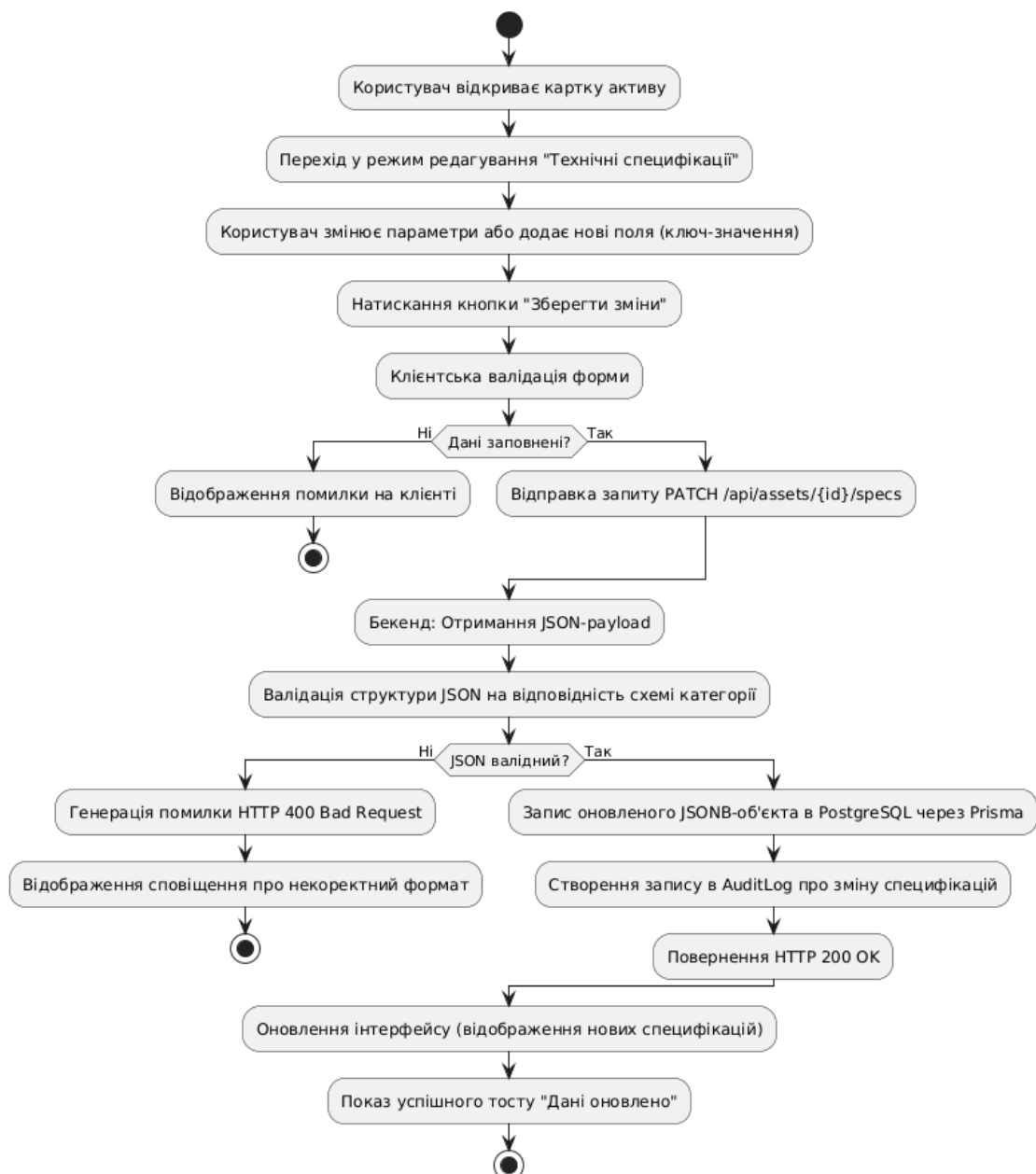


Рис. А.9 – Діаграма активності "Редагування технічних специфікацій"

Діаграма активності "Взаємодія з модулем ліцензій"

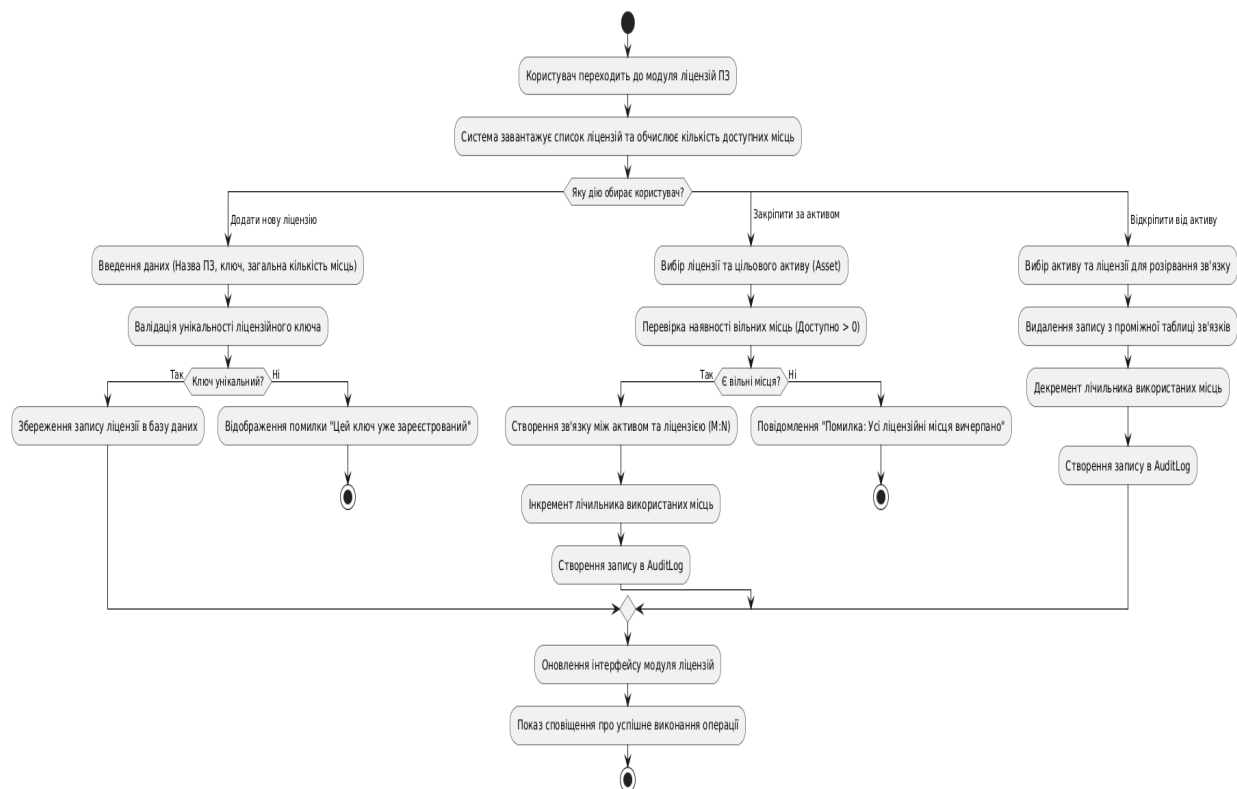


Рис. А.10 – Діаграма активності "Взаємодія з модулем ліцензій"

Діаграма активності "Процес списання активу"

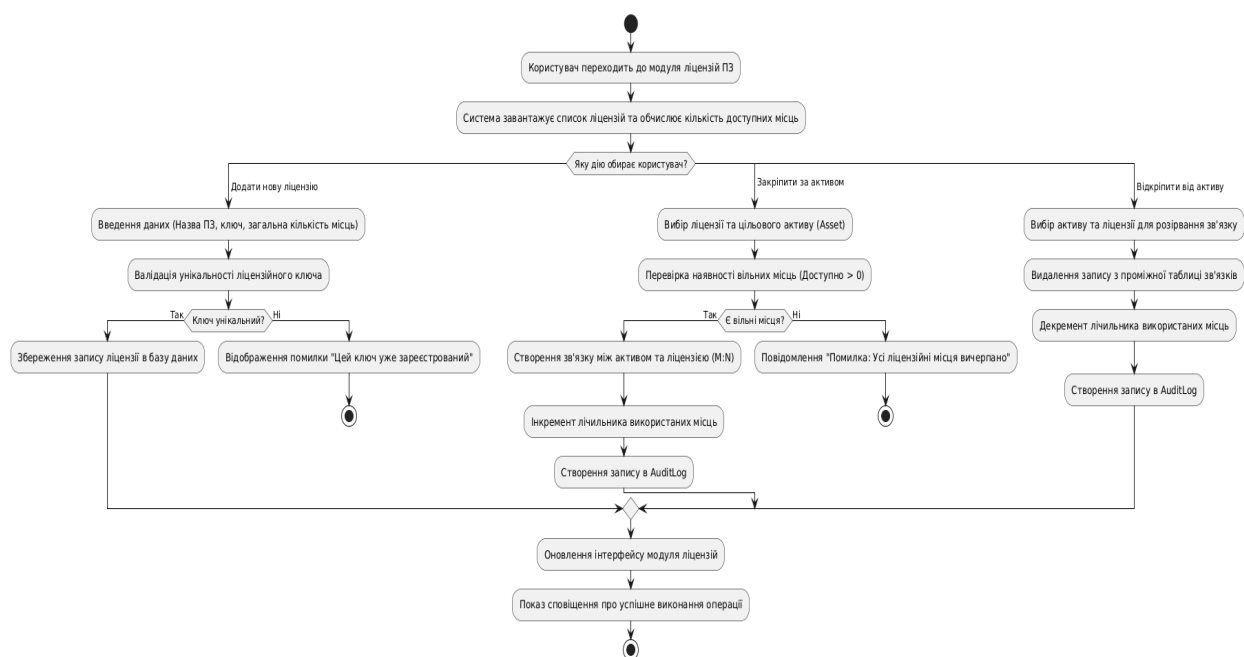


Рис. А.11 – Діаграма активності "Процес списання активу"

Фрагменти коду

Б.1 Схема бази даних

```
generator client {  
  
  provider = "prisma-client-js"  
  
}  
  
datasource db {  
  
  provider = "postgresql"  
  
  url      = env("DATABASE_URL")  
  
}  
  
model Asset {  
  
  id          String    @id @default(uuid())  
  
  name        String  
  
  category    String  
  
  status      String  
  
  serialNumber String    @unique  
  
  inventoryId String    @unique  
  
  price       Float  
  
  location    String  
  
  user        String  
  
  type        String?  
  
  brand       String?
```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						88
Змн.	Арк.	№ докум.	Підпис	Дата		


```

model      String?

specs      String?

createdAt  DateTime @default(now())

updatedAt  DateTime @updatedAt

@@map("assets")

}

model SystemSettings {

    id          Int      @id @default(1)

    adminName    String   @default("System Admin")

    adminEmail    String   @default("admin@itasset.com")

    currency     String   @default("UAH")

    assetPrefix   String   @default("ITA-")

    emailNotif    Boolean @default(true)

    slackNotif    Boolean @default(false)

    slackWebhook  String   @default("")

    maintenanceMode Boolean @default(false)

}

model Server {

    id          String   @id @default(uuid())

    name        String

    ip          String

    type        String

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						89
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    status      String

    cpu          String

    ram          String

    uptime      String

    createdAt   DateTime @default(now())

    @@map("servers")

}

```

```

model Log {

    id           String    @id @default(cuid())

    time         String

    actor        String

    type         String

    source       String

    text         String

    createdAt   DateTime @default(now())

    @@map("logs")

}

```

```

model Ticket {

    id           String    @id @default(uuid())

    title        String

    description  String?

}

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						90
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        status      String      @default("Відкрито")

        priority     String      @default("Середній")

        user         String

        createdAt     DateTime    @default(now())

        updatedAt     DateTime    @updatedAt

        @@map("tickets")

    }

```

```

model SecurityLog {

    id          String      @id @default(uuid())

    event       String

    ip          String

    device      String

    time        String

    status      String

    createdAt    DateTime    @default(now())

    @@map("security_logs")

}

```

```

model SecuritySettings {

    id          Int          @id @default(1)

    twoFactor    Boolean     @default(false)

    ldapEnabled  Boolean     @default(false)

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						91
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    apiKey      String    @default("")

    @@map("security_settings")
}

model User {
    id          String    @id @default(cuid())

    name        String

    email       String    @unique

    password    String

    role        String    @default("tech")

    createdAt   DateTime  @default(now())

    updatedAt   DateTime  @updatedAt
}

model License {
    id          String    @id @default(cuid())

    name        String

    softwareType String

    licenseKey   String?

    totalSeats   Int

    usedSeats    Int      @default(0)

    expirationDate DateTime?

    createdAt    DateTime  @default(now())
}

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						92
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        updatedAt      DateTime @updatedAt

    @@map("licenses")

}

model Employee {

    id          String    @id @default(uuid())

    firstName   String

    lastName    String

    email       String    @unique

    position    String

    department  String

    status      String    @default("Активний")

    createdAt   DateTime @default(now())

    updatedAt   DateTime @updatedAt

    @@map("employees")

}

```

Б.2 API / Server Actions

```

import { neon } from '@neondatabase/serverless';

import { NextResponse } from 'next/server';

import { sendSlackNotification } from '@lib/slack';

const sql = neon(process.env.DATABASE_URL!);

)

export async function PUT(req: Request, { params }: { params: any }) {

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		93

```

try {

    const resolvedParams = await params;

    const id = resolvedParams.id;

    const body = await req.json();

    const { name, status, category, brand, model, serial_number, specs, assignedTo
} = body;

    let userToStore = assignedTo || 'Не призначено';

    if (['retired', 'missing'].includes(status)) {

        userToStore = 'Не призначено';

    }

    const updatedAsset = await sql`

        UPDATE assets

        SET

            name = ${name},

            type = ${category || 'Інше'},

            status = ${status},

            category = ${category},

            brand = ${brand || null},

            model = ${model || null},

            "serialNumber" = ${serial_number || null},

            specs = ${specs || null},

            "user" = ${userToStore},

            "updatedAt" = NOW()

        WHERE id = ${id}

        RETURNING *;

    `;

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						94
Змн.	Арк.	№ докум.	Підпис	Дата		

```

if (updatedAsset.length === 0) {

    return NextResponse.json({ error: 'Актив не знайдено' }, { status: 404 });

}

const asset = updatedAsset[0];

try {

    const settingsResult = await sql`SELECT "slackNotif", "slackWebhook" FROM
settings LIMIT 1`;

    const settings = settingsResult[0];

    if (settings && settings.slackNotif && settings.slackWebhook) {

        const payload = {

            blocks: [

                {

                    type: 'section',

                    text: {

                        type: 'mrkdwn',

                        text: `🔔 *Оновлено актив:* ${asset.name}\n*Новий статус:*
\`${status}\`\n*Користувач:* ${userToStore}`

                    }

                }

            ]

        };

        await sendSlackNotification(settings.slackWebhook, payload);

    }

} catch (slackError) {

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						95
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        console.error('Помилка відправки Slack (PUT):', slackError);
    }

    const assetWithFrontendFields = {

        ...asset,

        assignedTo: asset.user === 'Не призначено' ? '' : (asset.user || ''),

        serial_number: asset.serialNumber

    };

    return NextResponse.json(assetWithFrontendFields);

} catch (error: any) {

    return NextResponse.json({ error: error.message || 'Failed to update asset'
}, { status: 500 });

}

}

export async function DELETE(req: Request, { params }: { params: any }) {

    try {

        const resolvedParams = await params;

        const id = resolvedParams.id;

        const deletedAsset = await sql`

            DELETE FROM assets

            WHERE id = ${id}

            RETURNING *;

        `;
    }

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						96
Змн.	Арк.	№ докум.	Підпис	Дата		


```

if (deletedAsset.length === 0) {

    return NextResponse.json({ error: 'Актив не знайдено' }, { status: 404 });

}

const asset = deletedAsset[0];

try {

    const settingsResult = await sql`SELECT "slackNotif", "slackWebhook" FROM
settings LIMIT 1`;

    const settings = settingsResult[0];

    if (settings && settings.slackNotif && settings.slackWebhook) {

        const payload = {

            blocks: [

                {

                    type: 'section',

                    text: {

                        type: 'mrkdwn',

                        text: `🔔 *Увага! Актив видалено з бази!*\n*Назва:*
${asset.name}\n*Категорія:* ${asset.category || 'Не вказано'}\n*ID:* \`${asset.id}\``

                    }

                }

            ]

        };

        await sendSlackNotification(settings.slackWebhook, payload);

    }

} catch (slackError) {

    console.error('Помилка відправки Slack (DELETE):', slackError);

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						97
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }

    return NextResponse.json({ message: 'Asset deleted successfully' });

  } catch (error: any) {

    return NextResponse.json({ error: error.message || 'Failed to delete asset'
}, { status: 500 });

  }

```

Б.3 Клієнтська частина / Інтерфейс користувача

```

'use client';

import React, { useState } from 'react';
import { Asset, AssetStatus, AssetType } from '@types/asset';
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from
"@/components/ui/table";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";

interface AssetTableProps {
  assets: Asset[];
  onDelete: (id: string) => void;
  onEdit: (asset: Asset) => void;
}

const getStatusLabel = (status: AssetStatus) => {
  switch (status) {
    case 'active': return 'Активний';
    case 'maintenance': return 'Обслуговування';
    case 'retired': return 'Списаний';
    case 'missing': return 'Відсутній';
    default: return status;
  }
};

export default function AssetTable({ assets = [], onDelete, onEdit }:
AssetTableProps) {
  const [searchTerm, setSearchTerm] = useState('');
  const [filterType, setFilterType] = useState<AssetType | 'all'>('all');
  const filteredAssets = assets.filter(asset => {
    const matchesSearch = (asset.name ||
'').toLowerCase().includes(searchTerm.toLowerCase()) ||
(asset.owner ||
'').toLowerCase().includes(searchTerm.toLowerCase());
    const matchesFilter = filterType === 'all' || asset.type === filterType;
    return matchesSearch && matchesFilter;
  });

  const handleExportCSV = () => {
    if (filteredAssets.length === 0) return;

    const headers = ['Назва активу', 'Серійний номер', 'Тип', 'Статус',
'Власник', 'Локація'];
    const rows = filteredAssets.map(asset => [
      asset.name,
      asset.serialNumber || asset.id,
      asset.type,

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		98

```

        getStatusLabel(asset.status),
        asset.owner,
        asset.location
    ]);

    const csvContent = '\uFEFF' + [headers.join(','), ...rows.map(e => e.map(val
=> ` "${val}"`).join(','))].join('\n');
    const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
    const url = URL.createObjectURL(blob);
    const link = document.createElement('a');
    link.setAttribute('href', url);
    link.setAttribute('download', `it_assets_export_${new
Date().toISOString().split('T')[0]}.csv`);
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
};

return (
  <div className="glass-panel p-6 overflow-hidden border-none">
    <div className="flex justify-between items-center mb-6 gap-4">
      { /* Пошук */ }
      <Input
        placeholder="Пошук за назвою або власником..."
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />
      { /* Кнопка експорту */ }
      <Button onClick={handleExportCSV} disabled={filteredAssets.length ===
0}>
        Експорт CSV
      </Button>
    </div>

    { /* Таблиця */ }
    <Table>
      <TableHeader>
        <TableRow>
          <TableHead>Назва активу</TableHead>
          <TableHead>Статус</TableHead>
          <TableHead>Власник</TableHead>
          <TableHead>Дії</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {filteredAssets.map((asset) => (
          <TableRow key={asset.id}>
            <TableCell>{asset.name}</TableCell>
            <TableCell>{getStatusLabel(asset.status)}</TableCell>
            <TableCell>{asset.owner}</TableCell>
            <TableCell>
              <Button onClick={() => onEdit(asset)}>Редагувати</Button>
              <Button onClick={() => onDelete(asset.id)}
variant="destructive">Видалити</Button>
            </TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  </div>
);
}

```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						99
Змн.	Арк.	№ докум.	Підпис	Дата		

Б.4 компонент навігації з рольовим розмежуванням доступу

```
'use client';

import React, { useState, useEffect } from 'react';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import { useSession, signOut } from 'next-auth/react';
import { LayoutDashboard, Monitor, Database, Key, Ticket, Users, Settings, Bell,
LogOut } from 'lucide-react';
import { motion, AnimatePresence } from 'framer-motion';

const menuItems = [
  { icon: LayoutDashboard, label: 'Дашборд', href: '/', allowedRoles: ['admin',
'tech', 'user'] },
  { icon: Database, label: 'ІТ-Активи', href: '/assets', allowedRoles: ['admin',
'tech', 'user'] },
  { icon: Key, label: 'Ліцензії та ПЗ', href: '/licenses', allowedRoles:
['admin', 'tech'] },
  { icon: Ticket, label: 'Служба підтримки', href: '/helpdesk', allowedRoles:
['admin', 'tech', 'user'] },
  { icon: Users, label: 'Співробітники', href: '/employees', allowedRoles:
['admin', 'tech'] },
  { icon: Settings, label: 'Налаштування', href: '/settings', allowedRoles:
['admin'] },
];

export default function Sidebar() {
  const pathname = usePathname();
  const { data: session, status } = useSession();
  const [showNotifications, setShowNotifications] = useState(false);
  const [notifications, setNotifications] = useState<any[]>([]);

  useEffect(() => {
    if (status === 'authenticated') {
      fetchNotifications();
      const interval = setInterval(fetchNotifications, 30000);
      return () => clearInterval(interval);
    }
  }, [status]);

  const fetchNotifications = async () => {
    try {
      const res = await fetch('/api/notifications');
      if (res.ok) {
        const data = await res.json();
        setNotifications(data);
      }
    } catch (error) {
      console.error('Не вдалося завантажити сповіщення:', error);
    }
  };

  if (pathname === '/login') return null;

  const userRole = (session?.user as any)?.role || 'guest';
  const filteredMenuItems = menuItems.filter(item =>
item.allowedRoles.includes(userRole));
  const unreadCount = notifications.filter(n => !n.read).length;

  return (
    <aside className="w-[260px] h-[calc(100vh-32px)] flex flex-col sticky top-4
z-50 glass-panel">
      <div className="p-8 flex items-center gap-3">
```

					ІПЗ.КР.Б – 121 – 26 – ПЗ	Арк.
						100
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        <Monitor className="text-primary" />
        <span className="text-xl font-extrabold text-transparent">IT-Asset
OS</span>
    </div>

    <nav className="flex-1 px-3 flex flex-col gap-2">
        {filteredMenuItems.map((item) => {
            const isActive = pathname === item.href;
            return (
                <Link key={item.href} href={item.href}>
                    <motion.div className={`flex items-center p-3 rounded-xl gap-3
${isActive ? "text-primary bg-primary/10" : "hover:bg-white/5"}`>
                        <item.icon size={20} />
                        <span className="text-[15px] font-medium flex-
1">{item.label}</span>
                    </motion.div>
                </Link>
            );
        })}
    </nav>

    {/* Панель профілю та керування сесією */}
    <div className="p-4 border-t border-border flex items-center gap-3">
        <div className="overflow-hidden flex-1">
            <p className="text-sm font-semibold truncate text-white">
                {session?.user?.name || "Завантаження..."}
            </p>
            <p className="text-xs text-muted-foreground truncate">
                {userRole.toUpperCase()}
            </p>
        </div>

        <button onClick={() => setShowNotifications(!showNotifications)}
className="p-2 relative">
            <Bell size={18} />
            {unreadCount > 0 && <span className="absolute top-1 right-1 w-2 h-2
bg-red-500 rounded-full" />}
        </button>

        <button onClick={() => signOut({ callbackUrl: '/login' })} className="p-
2 hover:text-red-400">
            <LogOut size={18} />
        </button>
    </div>
</aside>
);
}

```